

面向鲲鹏昇腾的软件优化

部门：计算技术开发部

作者：王欣亮

日期：2025-05-13



目录

1. 面向鲲鹏架构的软件优化

A. 代码优化

I. Cache与内存

II. 指令

B. 加速库使用

I. 数学库

II. MPI

III. 编译器

IV. KUPL

V. 其他加速库

2. 面向昇腾架构的软件优化

3. 案例分析 – 矩阵乘法

华为鲲鹏处理器芯片路标

Production Now



32*A57@ 16FF
2.1GHz
4*64bit DDR3/4
PCIe 3.0 /SAS3.0/10GE



16*A57@ 16FF
2.1GHz
2*64bit DDR3/4
PCIe 3.0 /SAS3.0/10GE

Production Now



32*A72 @ 16FF+
2.4GHz
2P
4*64bit DDR3/4
PCIe 3.0 /SAS3.0/10GE

Production Now



32/48/64*自研核V1
Up to 3GHz
4P, 8*64bit DDR4
RoCE v2
PCIe 4.0 /100GE



24/32*自研核V1
Up to 3GHz
2P, 4*64bit DDR4
RoCE v2
PCIe 4.0 /100GE

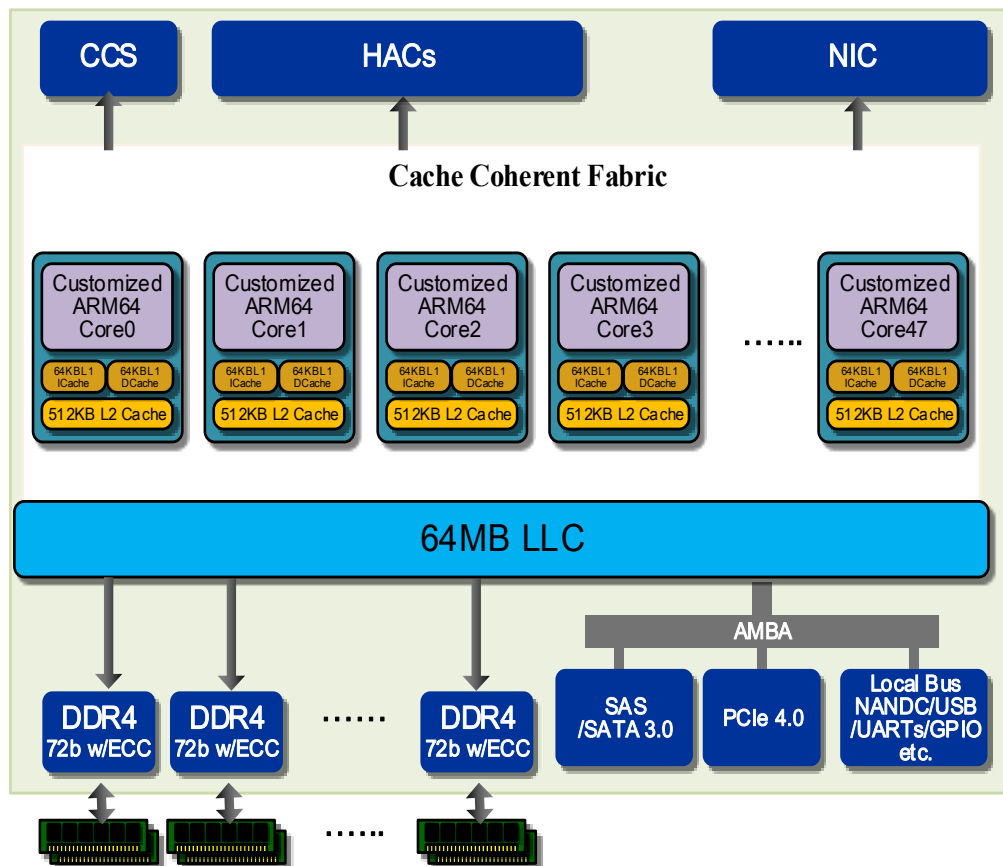
2015

2017

2019

Cache与内存

鲲鹏920芯片规格



➤集成最多64*自研核

➤指令集兼容ARMv8.2, 最高主频达3.0GHz

➤每核集成64KB L1 I/D 缓存

➤每核独享 512KB L2 缓存, 单芯片共享48-64MB L3缓存

➤8*DDR4控制器@2933MT/s

➤集成PCIe/SAS接口

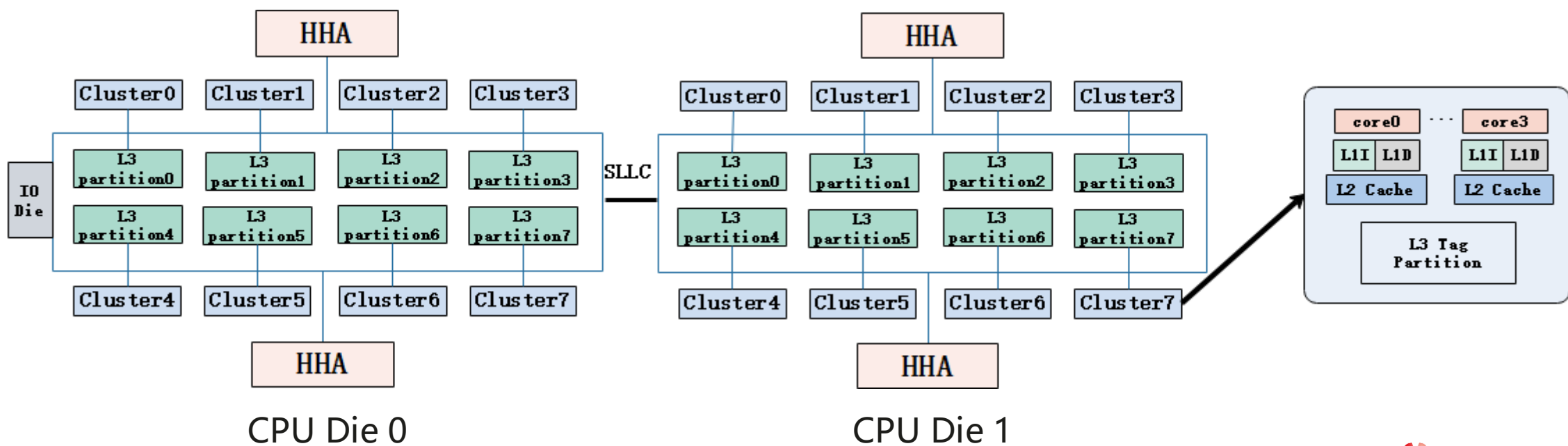
➤支持PCIe 4.0, 向下兼容PCIe 3.0/2.0/1.0

➤支持x16,x8,x4,x2,x1 PCIe 4.0, 集成20 PCIe控制器

➤支持16*SAS/SATA 3.0控制器

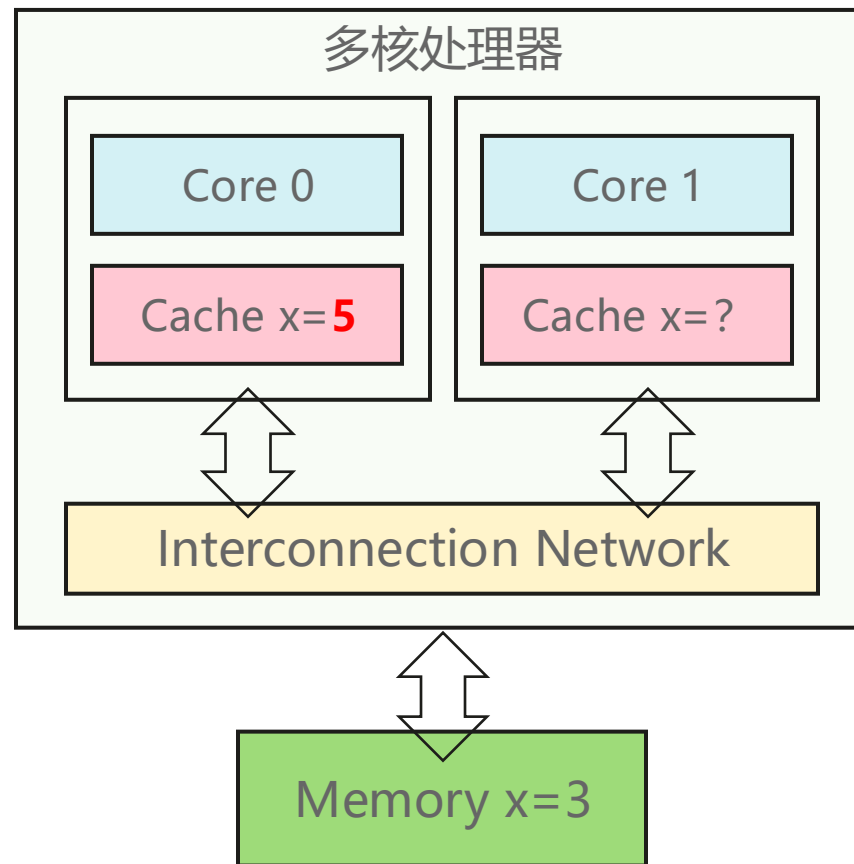
鲲鹏920 Uncore架构介绍

- 一颗Kunpeng920芯片包含两个CPU Die，一个IO Die；CPU Die主要包括Core以及Cache，IO Die 主要包括IO接口，Die间利用片上SLLC总线连接
- 每个CPU Die包含8个 Cluster，8个L3 Partition。
- LLC的管理方式分为Shared、Partition、Private三种，默认Partition方式
- 每个Cluster包含4个core，每个core有64KB大小的私有 L1 数据与指令Cache，512KB私有的L2 Cache，Cluster内还包含本L3 Partition的L3 Tag信息



鲲鹏920 Cache一致性介绍

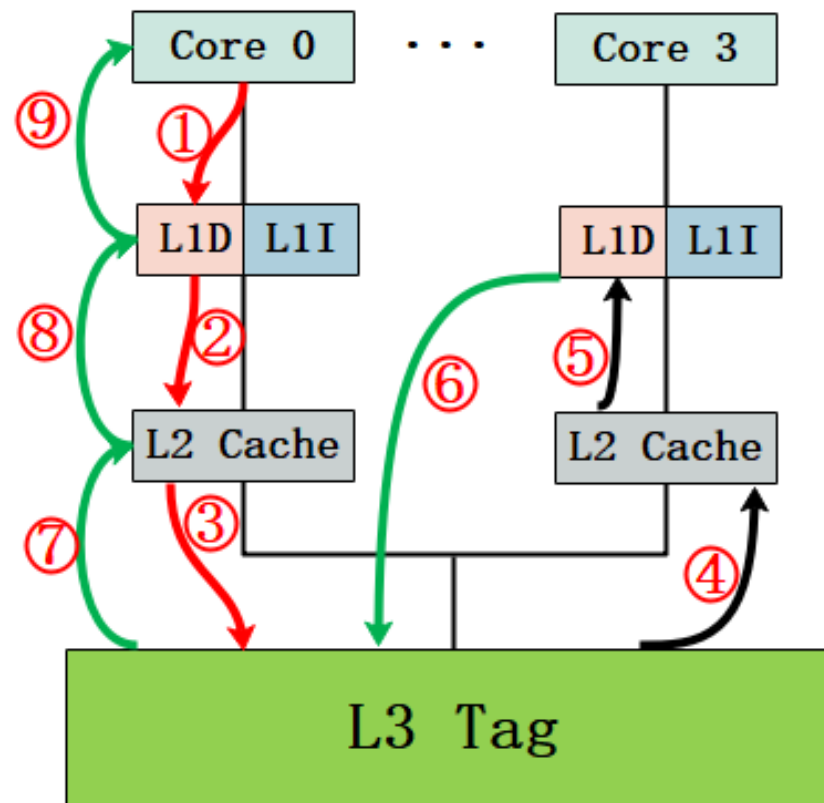
- Cache一致性问题：X为多线程共享数据，初始化为3。core0先将x读入core0私有cache中值为3，core1再将x读入core1的私有cache中值为3；之后core0将x更新为5，之后core1去读x值，将会读到旧值3，产生错误。
- 问题根因：由于多核私有Cache导致同一共享变量在系统中存在多个副本，当某一副本被修改时其他core将读到错误的数据



鲲鹏920 Cache一致性介绍

一、Cluster内Cache一致性处理流程

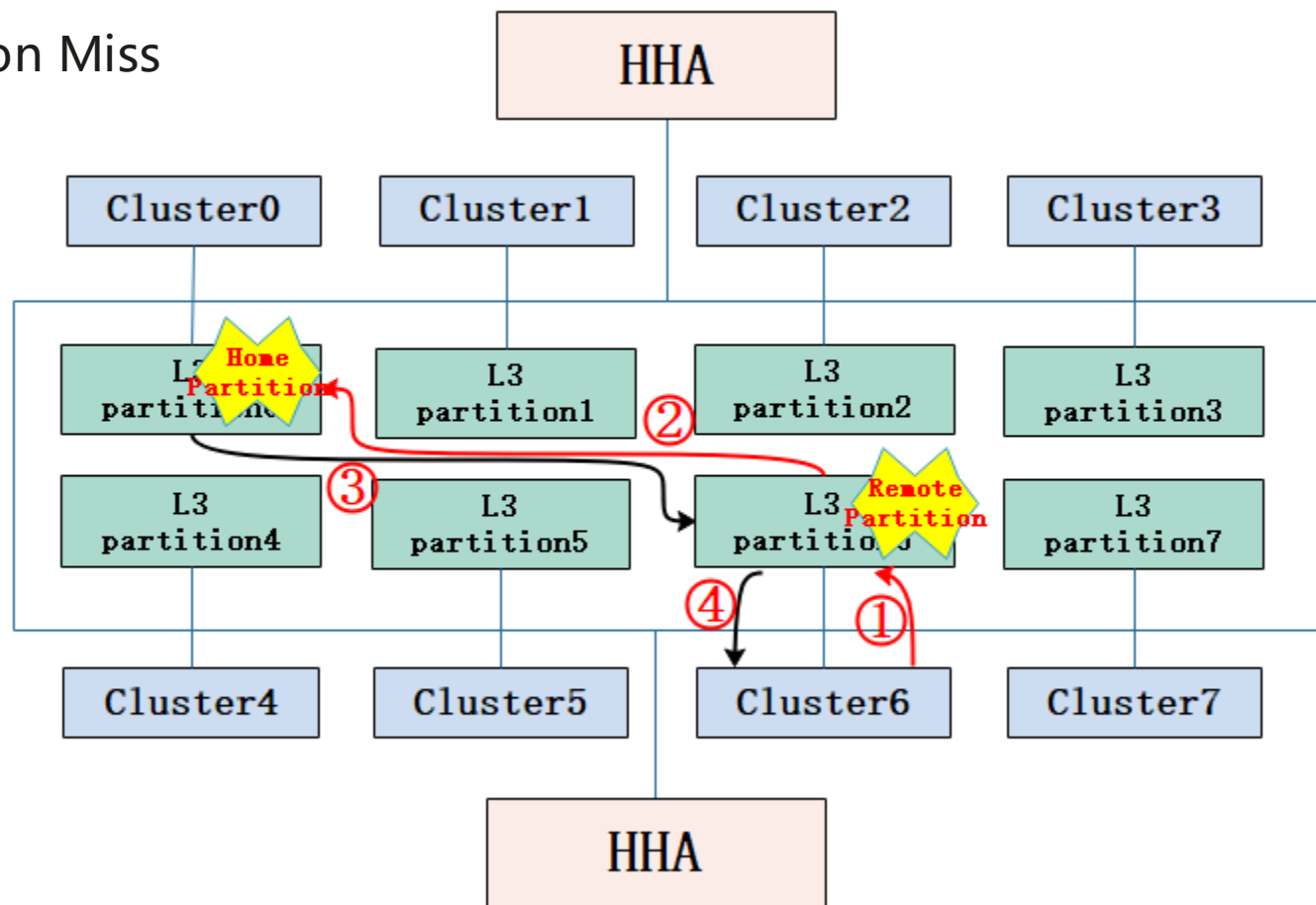
- 1、Core 0读数据 L1 L2 读 Miss
- 2、CA搜索L3 Tag以及目录信息，确定数据在Core3的Cache上
- 3、CA向Core3的Cache上发送数据读snoop
- 4、Core3的Cache将数据发送给L3 Tag上的CA，并将数据放到core0的Cache中，Cache状态变为S态
- 5、Cluster内读M态延迟大约为20ns



鲲鹏920 Cache一致性介绍

二、Cluster间一致性处理流程

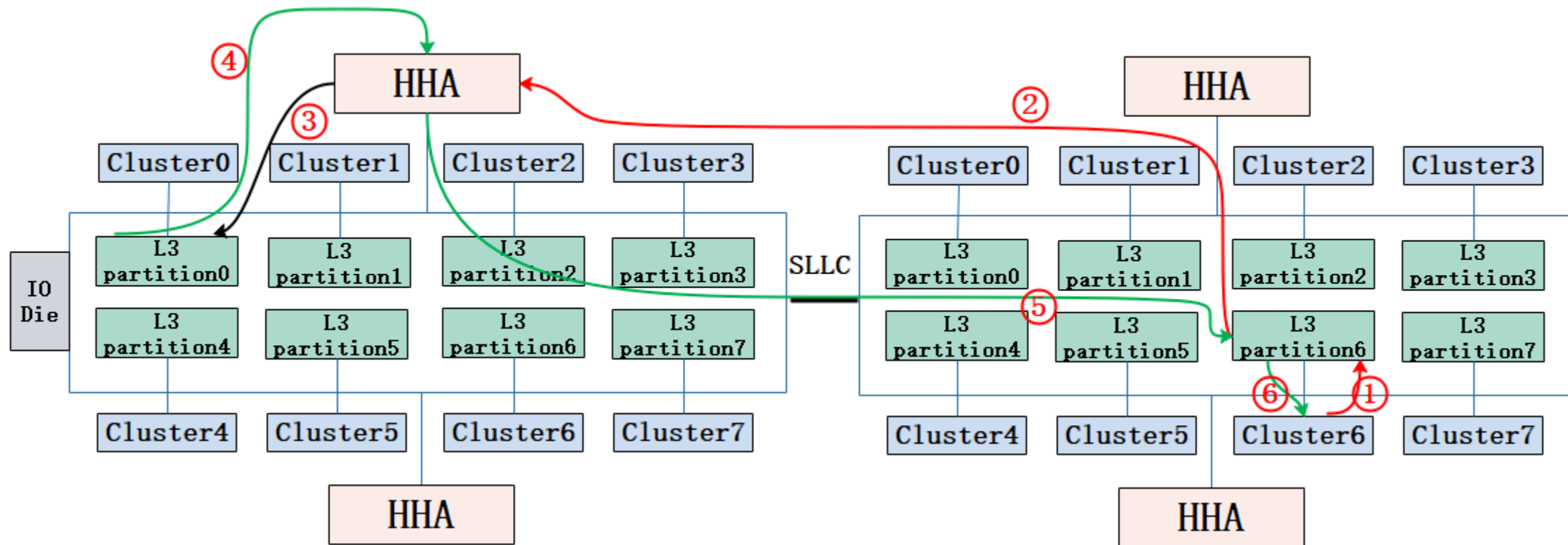
- 1、Cluster 6中的某个Core读取本地L3 partition Miss
- 2、L3 Partition 6 CA向请求数据的home Partition发送数据读snoop
- 3、Home Partition将数据发送给Partition 6
- 4、Cluster间读M态延迟大约为95ns



鲲鹏920 Cache一致性介绍

三、Die间一致性处理流程

- 1、Cluster 6中的某个Core读取本地L3 partition Miss
- 2、L3 Partition 6 CA根据数据的地址发现请求数据在另外一个Die上，向另外一个Die的HHA发送数据读请求
- 3、另外一个Die的HHA通过查询目录发现最新的数据在Partition 3上，向Partition 3发送数据读Snoop
- 4、Partition 3将数据发送给HHA，HHA将数据发送给请求数据的Partition 6
- 5、Die间读M态延迟大约为120ns



优化点1：多进程伪共享

多线程Cache伪共享能够导致没有必要的Cache Miss，从而导致性能下降

解决伪共享的办法通常利用空间减小时间，让每个数据元素独占一个Cache Line，这样当一个元素改变时，另一个元素不受影响

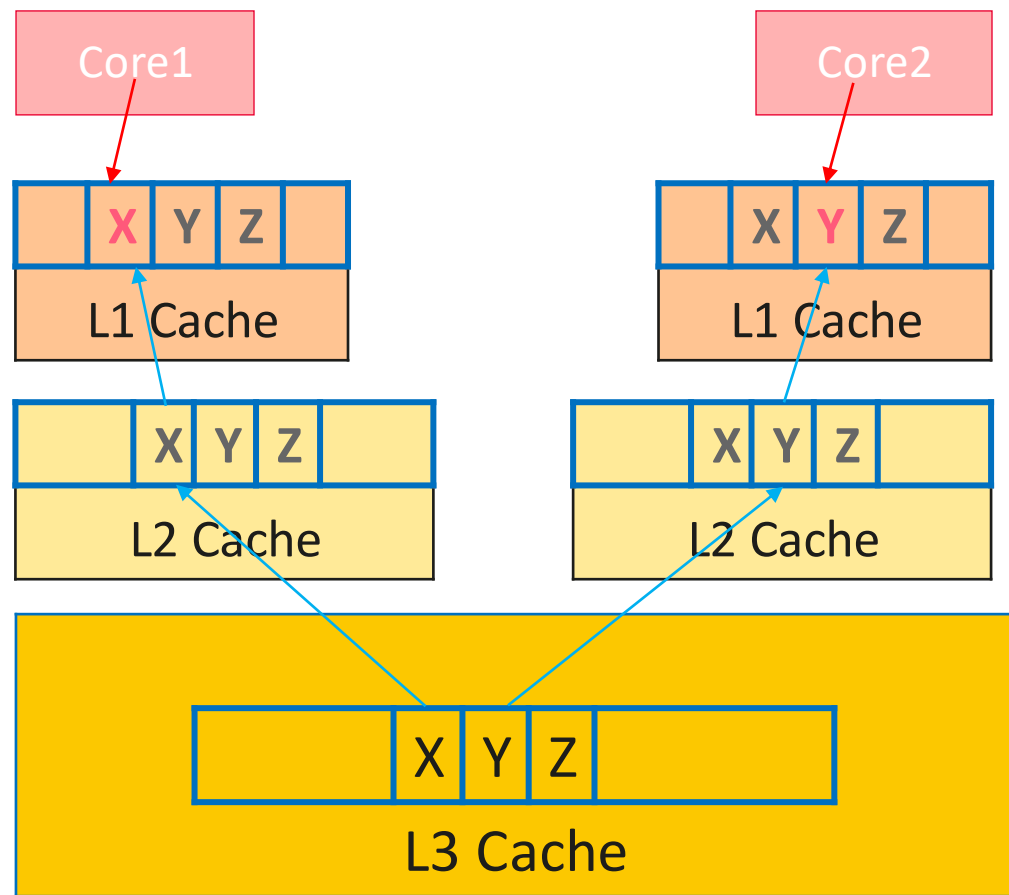
```
typedef struct Date_t {  
    long long int dateA;  
    long long int dateB;  
}tagDate;
```

有伪共享问题

```
typedef struct Data_t {  
    long long int dataA;  
    long long int padding[15];  
    long long int dataB;  
}tagDate;
```

无伪共享问题

消除伪共享后性能基本可以提升40%以上



两个不同的线程，操作同一CacheLine中的不同字节，产生了不是真正共享的伪共享

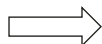
优化点2: Cache预取

Cache预取表示处理器要访问的数据, 提前预取到Cache中, 这样当处理器访问该数据时就可以直接从Cache中获取, 提高Cache命中率, 从而提升性能。

Cache预取分为两类, 一类是硬件预取, 由CPU中的Cache管理模块实现, 对程序员透明; 另一类是软件预取, 利用对应的预取指令在代码中实现数据的预取。(本次只介绍软件预取)

```
for (local_int_t i = 0; i < nrow; i++) {
    const double *const currentValues = A.matrixValues[i];
    const local_int_t *const currentColIndices = A.mtxIndL[i];
    const int currentNumberOfNonzeros = A.nonzerosInRow[i];
    const double currentDiagonal = matrixDiagonal[i][0]; // Current diagonal value
    double sum = rv[i]; // RHS value

    for (int j = 0; j < currentNumberOfNonzeros; j++) {
        local_int_t curCol = currentColIndices[j];
        sum -= currentValues[j] * xv[curCol];
    }
    sum += xv[i] * currentDiagonal; // Remove diagonal contribution from previous loop
    xv[i] = sum / currentDiagonal;
}
```

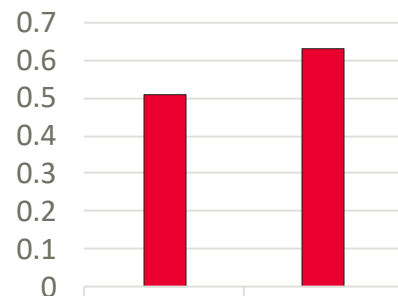


```
for (local_int_t i = 0; i < nrow; i++) {
    const double *const currentValues = A.matrixValues[i];
    const local_int_t *const currentColIndices = A.mtxIndL[i];
    const int currentNumberOfNonzeros = A.nonzerosInRow[i];
    const double currentDiagonal = matrixDiagonal[i][0]; // Current diagonal value
    double sum = rv[i]; // RHS value

    register local_int_t curCol, j;
    for (j = 0; j < currentNumberOfNonzeros - PREFETCH_OFFSET; j++) {
        __builtin_prefetch(&xv[currentColIndices[j + PREFETCH_OFFSET]], 0, 2);
        curCol = currentColIndices[j];
        sum -= currentValues[j] * xv[curCol];
    }
    for (; j < currentNumberOfNonzeros; j++) {
        curCol = currentColIndices[j];
        sum -= currentValues[j] * xv[curCol];
    }
    sum += xv[i] * currentDiagonal; // Remove diagonal contribution from previous loop
    xv[i] = sum / currentDiagonal;
}
```



HPCG软件预取性能对比



优化前 优化后

性能提升21.6%

基于perf top 定位到HPCG的热点代码段

软件预取优化

<https://developer.arm.com/documentation/101458/2010/Coding-best-practice/Prefetching-with---builtin-prefetch>

常见芯片查询网站

- <https://www.wikipedia.org/>
- <https://en.wikichip.org/wiki/WikiChip>
- <https://www.intel.com/content/www/us/en/ark.html>

DDR4	DDR4 2133: 17 GB/s DDR4 2400: 19.2 GB/s DDR4 2666: 21.3 GB/s DDR4 3200: 25.6 GB/s
DDR5	DDR5 4800: 38.4 GB/s DDR5 5200: 41.6 GB/s DDR5 5400: 43.2 GB/s DDR5 5600: 44.8 GB/s DDR5 6000: 48.0 GB/s DDR5 6400: 51.2 GB/s

<https://en.wikichip.org/wiki/hisilicon/kunpeng/920-6426>

Cache

Cache Organization

L1\$ 8 MiB

L1I\$ 4 MiB 64x64 KiB

L1D\$ 4 MiB 64x64 KiB

L2\$ 32 MiB

64x512 KiB

L3\$ 64 MiB

64x1 MiB

Memory controller

Integrated Memory Controller

Max Type

DDR4-2933

Supports ECC

Yes

Max Mem

2 TiB

Controllers

1

Channels

8

Width

64 bit

Max Bandwidth

190.7 GiB/s

Bandwidth

Single 23.84 GiB/s

Double 47.68 GiB/s

Quad 95.37 GiB/s

Octa 190.7 GiB/s

13 Huawei Proprietary - Restricted Distribution

STREAM Benchmark

<https://github.com/jeffhammond/STREAM>

Copy	$c[0:n] = a[0:n]$	1读1写
Scale	$c[0:n] = \text{scalar} * a[0:n]$	1读1写
Add	$c[0:n] = a[0:n] + b[0:n]$	2读1写
Triad	$c[0:n] = a[0:n] + \text{scalar} * b[0:n]$	2读1写

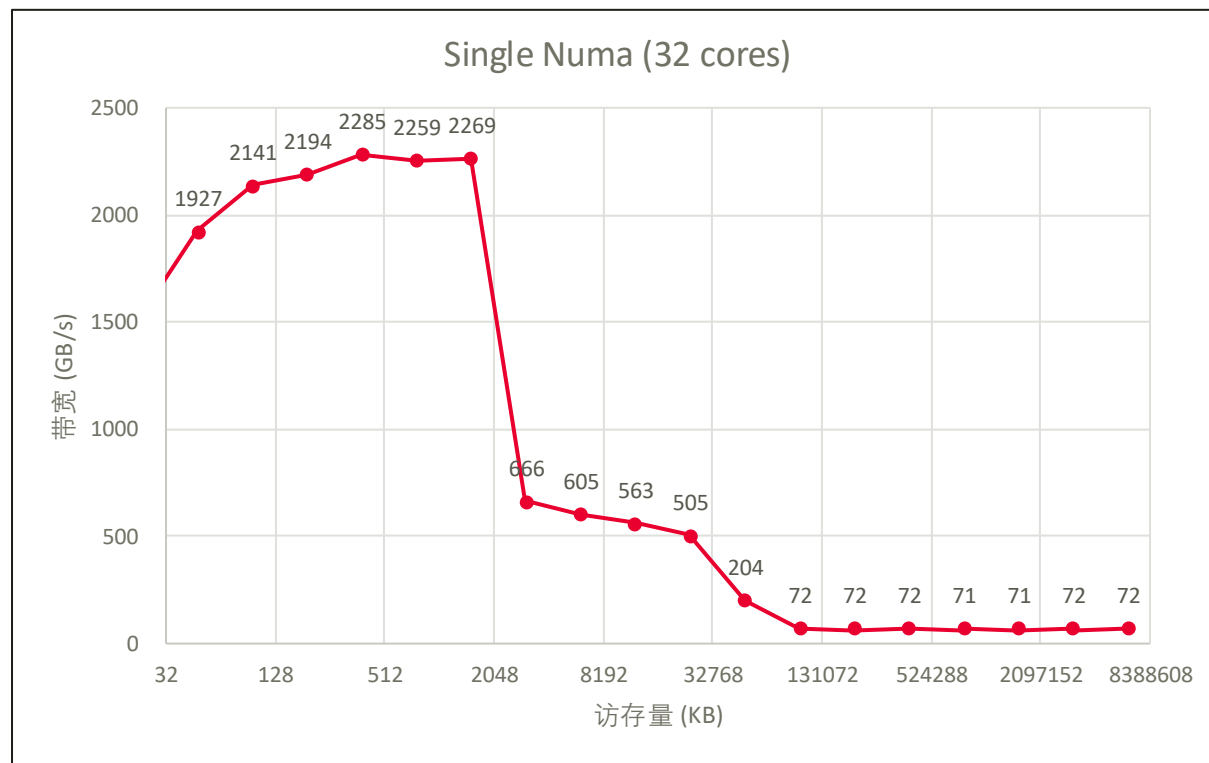
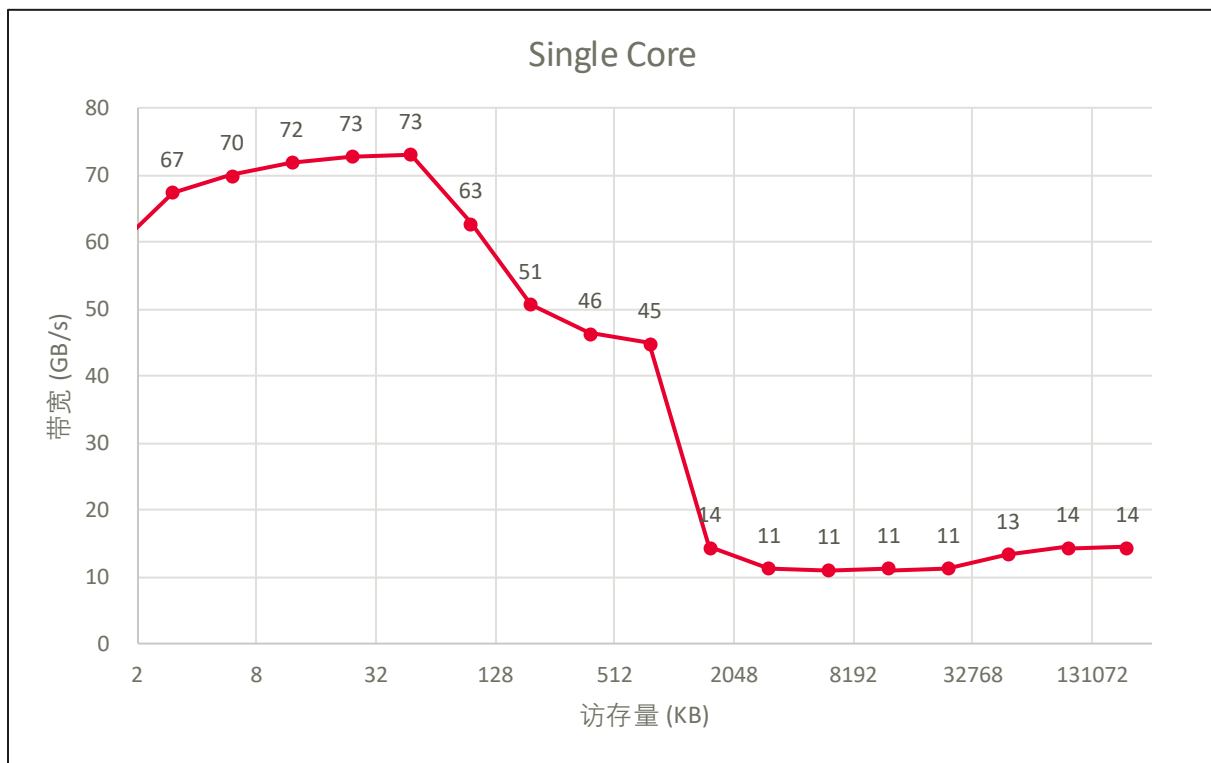
```
Number of Threads requested = 32
Number of Threads counted = 32
-----
Your clock granularity/precision appears to be 1 microseconds.
Each test below will take on the order of 24708 microseconds.
    (= 24708 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
-----
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
-----
Function      Best Rate MB/s  Avg time     Min time     Max time
Copy:         64879.6    0.024698    0.024661    0.024737
Scale:        66222.8    0.024236    0.024161    0.024286
Add:          66955.3    0.035895    0.035845    0.035935
Triad:        66728.1    0.037315    0.035967    0.047227
-----
```

```
Number of Threads requested = 64
Number of Threads counted = 64
-----
Your clock granularity/precision appears to be 1 microseconds.
Each test below will take on the order of 15738 microseconds.
    (= 15738 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
-----
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
-----
Function      Best Rate MB/s  Avg time     Min time     Max time
Copy:         90492.0    0.017730    0.017681    0.017768
Scale:        116532.7    0.013852    0.013730    0.013936
Add:          111472.8    0.021583    0.021530    0.021607
Triad:        112039.8    0.021534    0.021421    0.021766
-----
```

STREAM Benchmark 续

通过改造STREAM，可以测试Cache的吞吐

- 为了测试L1/L2 Cache带宽，需要用向量化访存



指令

指令集概述

- 指令：计算机执行某种操作的命令, 是计算机运行的最小单元, 由操作码 (Opcode) 和操作数 (Operands) 组成

高级语言程序 (c/c++)

↓ 编译器

汇编语言程序

↓ 汇编器

机器语言程序

```
int main() {  
    int d = 67;  
    return d;  
}  
  
push    %rbp  
mov     %rsp,%rbp  
movl    $0x43,-0x4(%rbp)  
mov     $0x0,%eax  
  
55  
48 89 e5  
c7 45 fc 43 00 00 00
```

```
movl    $0x43,-0x4(%rbp)
```

- X86:
- 将67立即数存入`rbp-4`指向的内存中
 - Movl: 1后缀说明操作数为32位
 - \$0x43为16进制67
 - `rbp`为基指针, 指向函数栈帧的顶部, `rbp-4`为指向内存的地址

```
mov     w0, 67  
str     w0, [sp, 12]
```

- 指令集差异

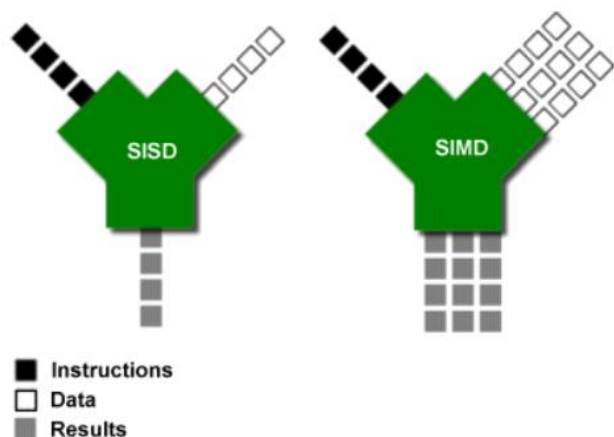
CISC复杂指令集 (X86)	RISC简单指令集 (ARM)
变长指令, 需要变长解码	定长指令, 固定格式解码
指令内存操作无限制	仅Load/Store指令进行访存
单条指令执行复杂操作	单条指令执行简单操作, 需要组合多条指令完成复杂操作

- ARM:
- 将67立即数存入w0寄存器, 再将W0中的数据存入到内存地址
 - 用store指令进行访存

ARM 指令集演进

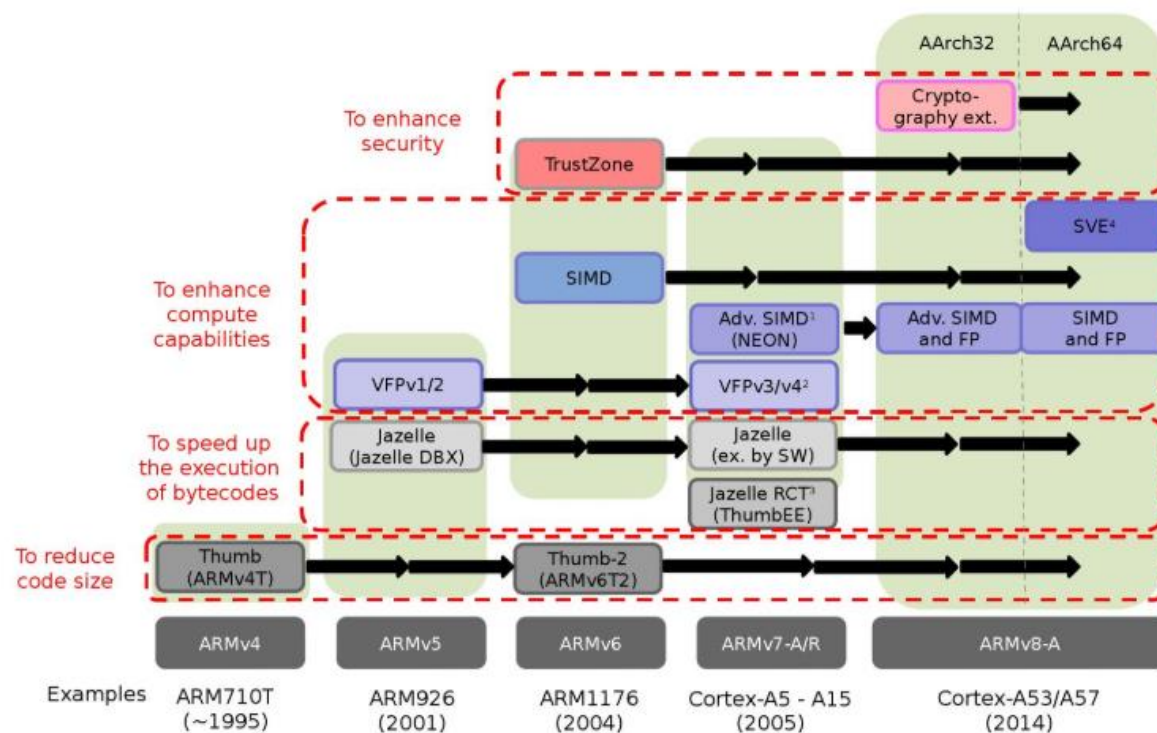
为什么指令集要演进？

- 多媒体处理：图像处理、视频编解码
- 机器学习和人工智能：矩阵乘法和加法运算
- SIMD：单条指令对多个数据同时进行相同操作的数据级并行化（DLP）技术
- SIMD加速条件：多路数据间无依赖、类型相同、计算相同这三个条件



概述

- VFP extension：允许处理器执行浮点运算，与intel X87 PFU指令功能相似
- NEON（Advance SIMD）：ARM的SIMD指令集
- SVE、SVE2: SVE2是SVE和Neon的超集
- SME：矩阵计算单元，在SVE、SVE2基础上对矩阵操作的增强

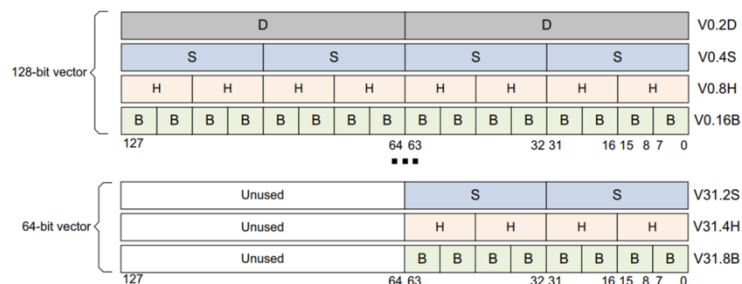


ARM 指令集差异

NEON指令

- 单精度、半精度、双精度、int8/16/32/64 类型
- 32个128bit 寄存器，支持64-bit/128-bit向量宽度
- 指令类型：Long/narrow/wide/normal
- 编译参数：-ftree-vectorize -O3

Figure 7-4: Neon registers and lanes

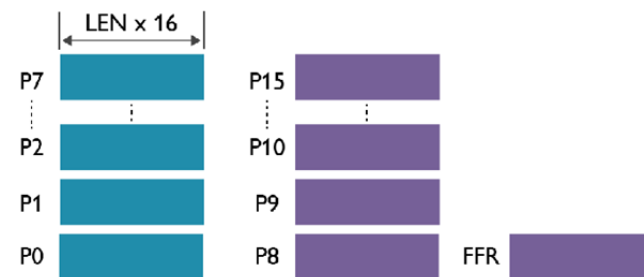
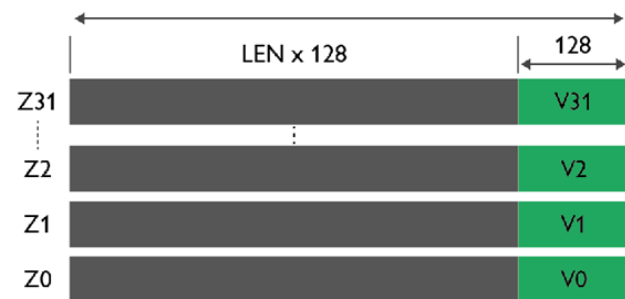


寄存器关系:

- 通用寄存器 31* 64bit
- NEON和FP共用 32*128bit向量寄存器
- SVE 32* 128bit Z寄存器，扩展NEON的V寄存器

SVE指令

- 16位半精度、32位单精度浮点、64位双精度和整数运算
- 寄存器:
 - 32个向量寄存器 (Z0-Z31)
 - 16个predicate寄存器 (判断元素有效性, 仅低1bit有效)
 - FFR (首次故障寄存器)
- 向量长度可以从128 bit到2048 bit, 有硬件决定
- 编译参数: -march=armv8+sve+sve2



NEON 汇编代码格式

<prefix>

<op>

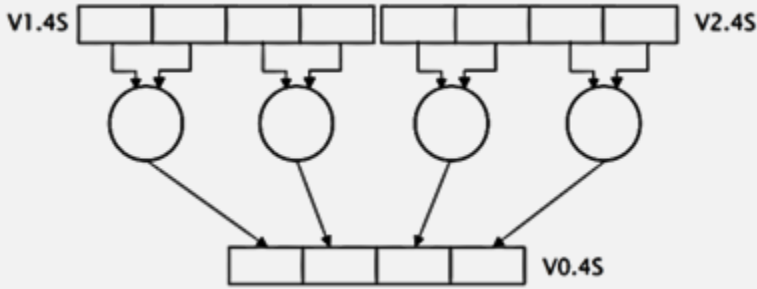
<suffix>

Vd.<T>

Vn.<T>

Vm.<T>

- <prefix> 前缀：S/U/F/P分别表示有符号整数/无符号整数/浮点/布尔类型
- <op> 操作符：如ADD、SUB、MUL等
- <suffix> 后缀：用于一些特殊操作，如(P)airwise操作（对向量内相邻数据两两操作），V（跨所有数据通道操作），L/N/W/L2/N2/W2：表示数据长度的变化，2在宽指令/窄指令中操作数据高位部分
- <T> 数据类型，通常是8B/16B/4H/8H/2S/4S/2D等。B=8bit；H=16bit；S=32bit；D=64bit

指令	解释
mul v0.8b, v0.8b, v1.8b	将v0和v1中的8路8bit数相乘，结果存入v0
fadd v0.4s, v0.4s, v1.4s	将v0和v1中的4路32bit数相加，结果存入v0
addp v0.4s, v1.4s, v2.4s	将v1和v2中各4路32bit数分别两两相加，结果拼合到v0 

saddl2 v0.4s, v1.8h, v2.8h
L：输出向量为输入向量的2倍
2：表示寄存器高64bit有效

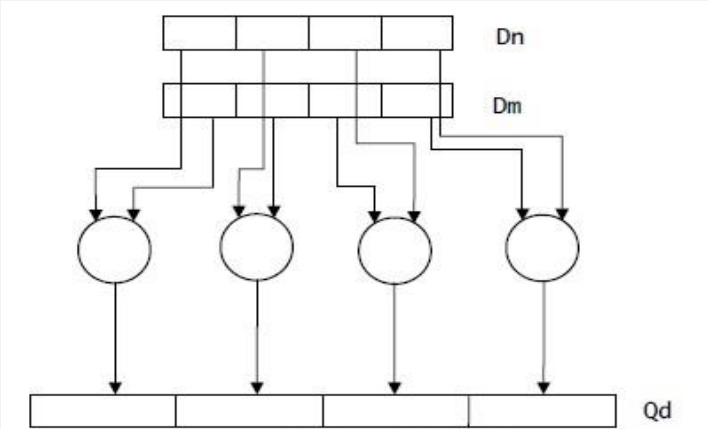


Figure 7-5 NEON long instructions

NEON 指令

Load/store指令

LD1 (single structure)

Load one single-element structure to one lane of one register. This instruction loads a single-element structure from memory and writes the result to the specified lane of the SIMD&FP register without affecting the other bits of the register.
Depending on the settings in the [CPACR_EL1](#), [CPTR_EL2](#), and [CPTR_EL3](#) registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: [No offset](#) and [Post-index](#)

No offset

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	0	1	0	1	0	0	0	0	0	0	x	x	0	S	size	Rn						Rt				
L										R	opcode																				

8-bit (opcode == 000)

LD1 { <Vt>.B }, [<Xn|SP>]

将8位的数据从通用寄存器Xn中移动到128位Neon/SVE向量寄存器的第index个数据中

ST1 (single structure)

Store a single-element structure from one lane of one register. This instruction stores the specified element of a SIMD&FP register to memory.
Depending on the settings in the [CPACR_EL1](#), [CPTR_EL2](#), and [CPTR_EL3](#) registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: [No offset](#) and [Post-index](#)

No offset

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	0	1	0	0	0	0	0	0	0	0	x	x	0	S	size	Rn						Rt				
L										R		opcode																			

8-bit (opcode == 000)

ST1 { <Vt>.B }, [<Xn|SP>]

将8位的数据从NEON/向量寄存器的第index个数据中存入到内存中

数据处理指令

FMOV (general)

Floating-point Move to or from general-purpose register without conversion. This instruction transfers the contents of a SIMD&FP register to a general-purpose register, or the contents of a general-purpose register to a SIMD&FP register.
Depending on the settings in the [CPACR_EL1](#), [CPTR_EL2](#), and [CPTR_EL3](#) registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	0	0	1	1	1	1	0	ftype	1	0	x	1	1	x	0	0	0	0	0	0	0	Rn						Rd			
rmode										opcode																					

向量寄存器以及通用寄存器间数据移动
FMOV <Wd>, <Hn>
FMOV <Vd>.D[1], <Xn>

逻辑运算指令

AND (vector)

Bitwise AND (vector). This instruction performs a bitwise AND between the two source SIMD&FP registers, and writes the result to the destination SIMD&FP register.
Depending on the settings in the [CPACR_EL1](#), [CPTR_EL2](#), and [CPTR_EL3](#) registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	Q	0	0	1	1	1	0	0	0	1	Rm						0	0	0	1	1	1	Rn						Rd			
size																																

AND <Vd>.<T>, <Vn>.<T>, <Vm>.<T>

向量逻辑与指令，V2 = V0 & V1

NEON 指令

数据处理

FADD (vector)

Floating-point Add (vector). This instruction adds corresponding vector elements in the two source SIMD&FP registers, writes the result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

This instruction can generate a floating-point exception. Depending on the settings in *FPCR*, the exception results in either a flag being set in *FPSR* or a synchronous exception being generated. For more information, see *Floating-point exception traps*.

Depending on the settings in the *CPACR_EL1*, *CPTR_EL2*, and *CPTR_EL3* registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: [Half-precision](#) and [Single-precision and double-precision](#)

Half-precision (FEAT_FP16)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	1	0	0	1	0	Rm				0	0	0	1	0	1	Rn				Rd						
U																															

FADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>

FDIV (vector)

Floating-point Divide (vector). This instruction divides the floating-point values in the elements in the first source SIMD&FP register, by the floating-point values in the corresponding elements in the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in *FPCR*, the exception results in either a flag being set in *FPSR*, or a synchronous exception being generated. For more information, see *Floating-point exception traps*.

Depending on the settings in the *CPACR_EL1*, *CPTR_EL2*, and *CPTR_EL3* registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: [Half-precision](#) and [Single-precision and double-precision](#)

Half-precision (FEAT_FP16)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	1	0	1	1	1	0	0	1	0	Rm				0		0	1	1	1	1	Rn				Rd					

FDIV <Vd>.<T>, <Vn>.<T>, <Vm>.<T>

FSUB (vector)

Floating-point Subtract (vector). This instruction subtracts the elements in the vector in the second source SIMD&FP register, from the corresponding elements in the vector in the first source SIMD&FP register, places each result into elements of a vector, and writes the vector to the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in *FPCR*, the exception results in either a flag being set in *FPSR*, or a synchronous exception being generated. For more information, see *Floating-point exception traps*.

Depending on the settings in the *CPACR_EL1*, *CPTR_EL2*, and *CPTR_EL3* registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: [Half-precision](#) and [Single-precision and double-precision](#)

Half-precision (FEAT_FP16)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	1	0	1	1	0					Rm		0	0	0	1	0	1									Rd
U																															

FSUB <Vd>.<T>, <Vn>.<T>, <Vm>.<T>

FMADD

Floating-point fused Multiply-Add (scalar). This instruction multiplies the values of the first two SIMD&FP source registers, adds the product to the value of the third SIMD&FP source register, and writes the result to the SIMD&FP destination register.

A floating-point exception can be generated by this instruction. Depending on the settings in *FPCR*, the exception results in either a flag being set in *FPSR*, or a synchronous exception being generated. For more information, see *Floating-point exception traps*.

Depending on the settings in the *CPACR_EL1*, *CPTR_EL2*, and *CPTR_EL3* registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	1	1	1	ftype	0					Rm		0			Ra											Rd	
o1																o0															

Half-precision (ftype == 11)
(FEAT_FP16)

FMADD <Hd>, <Hn>, <Hm>, <Ha>

NEON intrinsic汇编代码格式

指令格式

v<mod> <opname> <shape> <flags> _<type>

- <mod>前缀: Q/H/D/R/P分别表示饱和计算/饱和计算/加倍计算/舍入计算/pairwise计算
- <opname>操作符: 如ADD、SUB、MUL等
- <shape>后缀: L/N/W; _high: 与L/N搭配使用, 表示高64bit有效; _n: 表示有标量参与向量计算; _lane: 向量中某个通道参与向量计算
- <flags>: q表示使用128bit, 否则使用64bit向量
- <type>: 单个通道的数据类型

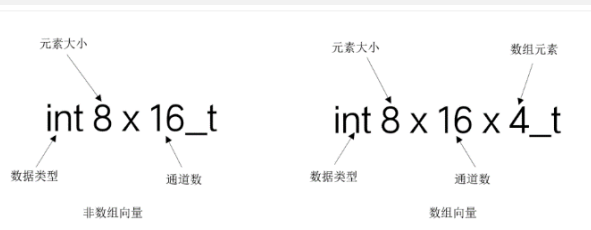
数据格式

非数组向量格式:

<type> <size>x<number_of_lanes>_t

数组向量格式:

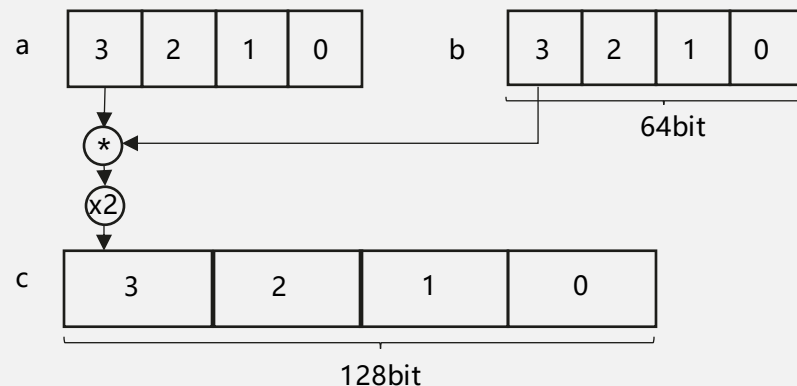
<type> <size>x<number_of_lanes>x<length_of_array>_t



指令示例:

// a乘b的结果扩大一倍, 最后做饱和操作

```
int32x4_t vqdmull_s16(int16x4_t a, int16x4_t b);
```



// 将a, b向量的相邻数据进行两两和操作

```
int8x8_t vpadd_s8(int8x8_t a, int8x8_t b);
```

// 将a, b向量的相邻数据进行两两和操作

```
int8x16_t vpaddq_s8(int8x16_t a, int8x16_t b);
```

// 取向量 v 中下标为 lane 的元素与向量 a 做乘法计算

```
int16x4_t vmul_lane_s16(int16x4_t a, int16x4_t v, const int lane);
```

SVE 汇编代码格式

$$\langle \text{opcode} \rangle \{ \langle \text{Zt} \rangle . \langle \text{T} \rangle \}, \langle \text{Pg} \rangle / \{ \text{M} | \text{Z} \}, \langle \text{Zn} \rangle . \langle \text{T} \rangle, \langle \text{Zm} | \text{Imm} \rangle . \langle \text{T} \rangle$$

由操作码(operation code)、目标寄存器(destination register)、谓词寄存器(predicate register) (如果指令支持谓词掩码) 和输入运算符组成。操作码前缀带f, 表明浮点运算

LDF1D {<Zt>.D}, <Pg>/Z, [<Xn | SP>, <Zm>.D, LSL #3]

矢量化加载指令，其中：

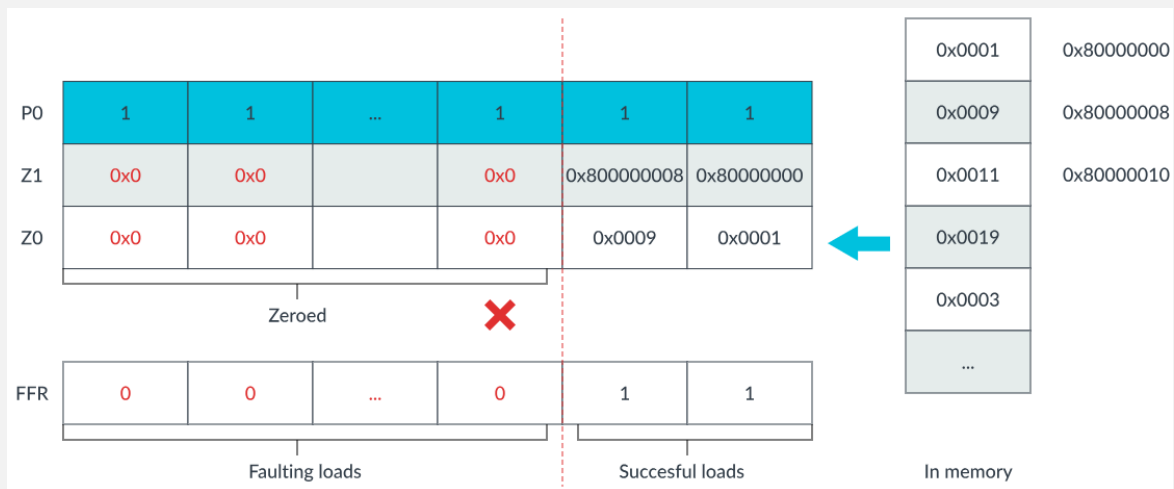
- $\langle Zt \rangle$ 是寄存器 Z0-Z31
- $\langle Zt \rangle.D$ 和 $\langle Zm \rangle.D$ 指定目标和操作数的数据类型
- $\langle Pg \rangle$ 是谓词寄存器 P0-P15
- $\langle Pg \rangle / Z$ 指示归零操作
- $Xn|SP$ 表示源操作数的基地址, Xn 为通用寄存器, SP 为栈指针寄存器
- $\langle Zm \rangle$ 指定集合加载地址模式的偏移量

LDF1D Z0.D, P0/Z, [Z1.D, #0]

解释：

通过LDFF1D指令从Z1存储的内存地址加载数据到Z0中。

FFR寄存器会将无效加载的元素位设置为0



SVE 汇编代码格式

<opcode> {<Zt>.<T>}, <Pg>/{M|Z}, <Zn>.<T>, <Zm|Imm>.<T>

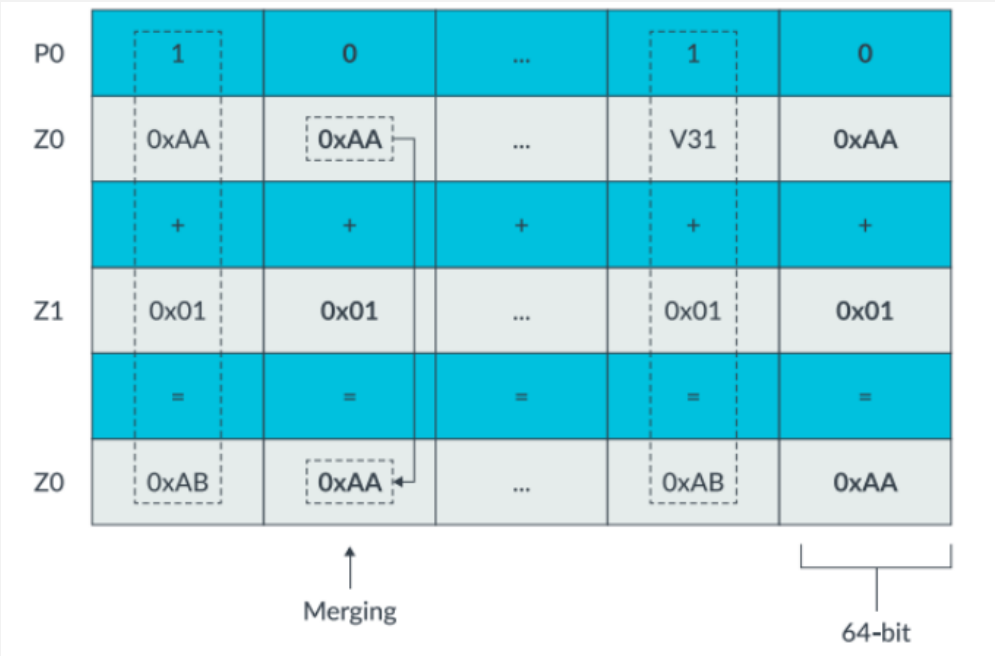
由操作码(operation code)、目标寄存器(destination register)、谓词寄存器(predicate register)（如果指令支持谓词掩码）和输入运算符组成。操作码前缀带f，表明浮点运算

ADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>

矢量化相加运算（Zdn和Zm寄存器相加），其中：

- <Pg>/M指示合并操作
- <Zdn>既是目标寄存器，也是输入运算符之一

ADD Z0.D, P0/M, Z0.D, Z1.D



SVE 汇编代码格式

```
LD1SB Z0.S, P0/Z, [Z1.S]
```

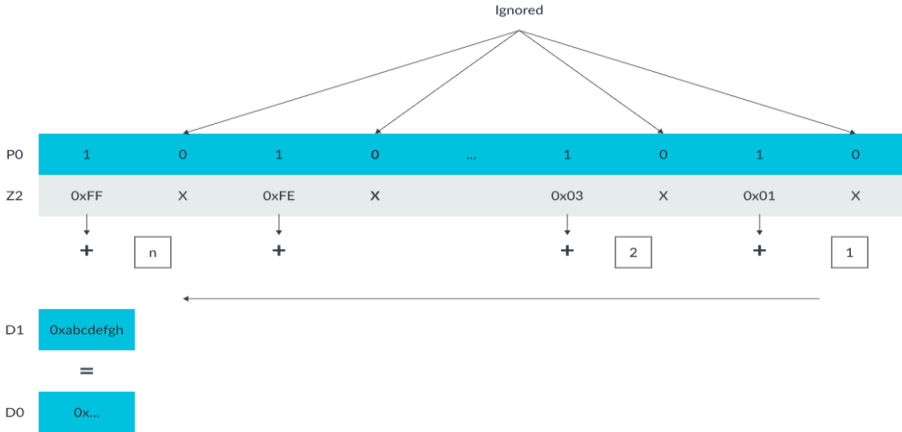
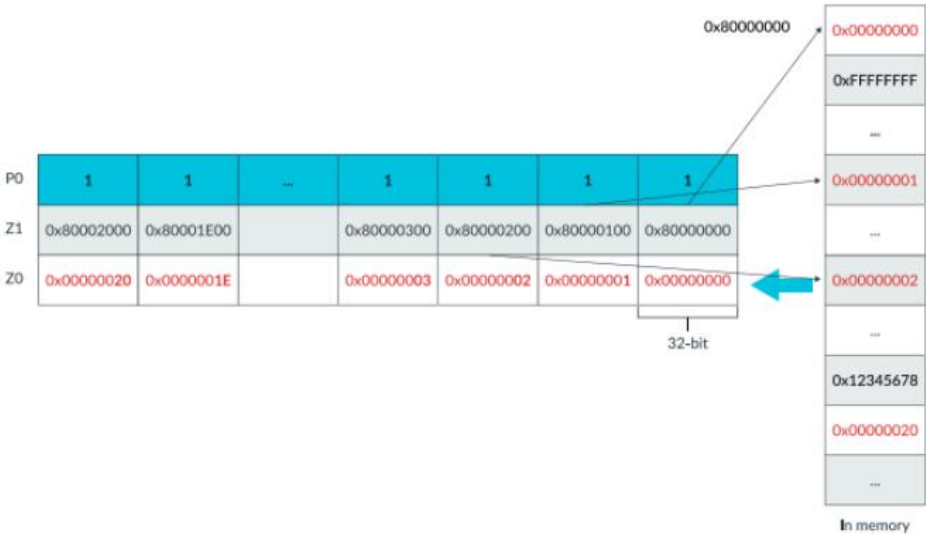
解释：
从寄存器Z1 生成的32位内存地址，加载有符号数到Z0寄存器中，
P0指示那些活动位需要加载

```
WHILELO <Pd>.<T>, <R><n>, <R><m>  
WHILELO P0.S, x8, x9  
B.FIRST Loop_start
```

循环控制指令，其中：
P0寄存器从最低编号的元素开始为真，当第一个无符号标量 X8 操作数的值高于第二个标量操作数 X9时，此后为假，直到最高编号的元素。

```
FADDA D0, P0/M, D1, Z2.D
```

解释：
将D1和Z2.D的有效元素（ P0指示的活动位）相加。并将结果放入标量



SVE 汇编代码格式

Load/store指令

LD1W (scalar plus immediate)

Contiguous load unsigned words to vector (immediate index)

Contiguous load of unsigned words to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 which is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.
It has encodings from 2 classes: [32-bit element](#) and [64-bit element](#)

32-bit element

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	1	0	1	0	0	imm4	1	0	1	Pg						Rn						Zt			
dtype<3:1>dtype<0>																															

LD1W { <Zt>.*S* }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]

从内存中加载单个字宽度的数据到指定的SVE向量寄存器中

ST1W (scalar plus immediate)

Contiguous store words from vector (immediate index)

Contiguous store of words from elements of a vector register to the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 which is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements are not written to memory.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	1	0	size	0	imm4	1	1	1	Pg						Rn						Zt				
msz<1>msz<0>																															

ST1W { <Zt>.<T> }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]

从指定的SVE向量寄存器内的数据存储在单个字宽度的数据到内存中；

逻辑运算指令

AND (predicates)

Bitwise AND predicates

Bitwise AND active elements of the second source predicate with corresponding elements of the first source predicate and place the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. Does not set the condition flags.
This instruction is used by the alias [MOV \(predicate, predicated, zeroing\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	0	0	0	0	Pm	0	1	Pg	0	Pn	0	Pd												
S																															

AND <Pd>.*B*, <Pg>/Z, <Pn>.*B*, <Pm>.*B*

与操作
根据谓语寄存器控制对比对象

数据传输和位移

ASR (immediate, unpredicated)

Arithmetic shift right by immediate (unpredicated)

Shift right by immediate, preserving the sign bit, each element of the source vector, and place the results in the corresponding elements of the destination vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	tszh	1	tszl	imm3	1	0	0	1	0	0	Zn						Zd							
U																															

ASR <Zd>.<T>, <Zn>.<T>, #<const>

算术右移操作；
空出的位由符号位填充，用于保存数值符号的场景使用

FADD (vectors, unpredicated)

Floating-point add vector (unpredicated)

Add all floating-point elements of the second source vector to corresponding elements of the first source vector and place the results in the corresponding elements of the destination vector. This instruction is unpredicated.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	Zm				0	0	0	0	0	0	Zn				Zd							

FADD <Zd>.<T>, <Zn>.<T>, <Zm>.<T>

<Zd>.<T> = <Zn>.<T> + <Zm>.<T>

FADDV

Floating-point add recursive reduction to scalar

Floating-point add horizontally over all lanes of a vector using a recursive pairwise reduction, and place the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as +0.0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	0	0	0	0	0	0	0	0	1	Pg	Zn				Vd							

FADDV <V><d>, <Pg>, <Zn>.<T>

将Zn 向量中的数据按照递归方式，进行两两相加归，最终获得标量值，将结果存储在标量寄存器中

FMLA (vectors)

Floating-point fused multiply-add vectors (predicated), writing addend [Zda = Zda + Zn * Zm]

Multiply the corresponding active floating-point elements of the first and second source vectors and add to elements of the third source (addend) vector without intermediate rounding. Destructively place the results in the destination and third source (addend) vector. Inactive elements in the destination vector register remain unmodified.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	1	Zm				0	0	0	Pg	Zn				Zda									

N op

FMLA <Zda>.<T>, <Pg>/M, <Zn>.<T>, <Zm>.<T>

对向量数据进行逐元素乘法操作,再与目的向量数据相加, inactive的元素保持不变

<Zda>.<T> = <Zda>.<T>+ <Zn>.<T> * <Zm>.<T> (if <Pg>/M is true)

FDIV

Floating-point divide by vector (predicated)

Divide active floating-point elements of the first source vector by corresponding floating-point elements of the second source vector and destructively place the quotient in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	0	1	1	0	1	1	0	0	Pg	Zm				Zdn								

FDIV <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>

<Zd>.<T> = <Zn>.<T> / <Zm>.<T>



SVE intrinsic汇编代码格式

指令格式

`svbase[disambiguator][_type0][_type1]...[predication]`

- base 操作符：如ADD、SUB、MUL等
- disambiguator：区分相同基本操作的不同变体
- typeN：向量和 P 寄存器的类型
- predication：指定了非活动元素的处理方式

数据格式

`sv<datatype><datasize>_t`

类型：

`svfloat64_t, svfloat16_t, svuint32_t`

指令示例：

- 基础计算

```
svadd_z(pg, sv_vec1, sv_vec2)
```

```
svadd_m(pg, sv_vec1, sv_vec2)
```

```
svadd_x(pg, sv_vec1, sv_vec2)
```

```
svadd_x(pg, sv_vec1, x)
```

// _z: 将pg为false的值置0, _m: 将pg为false的值保持

原值, _x: 将pg为false的位置, 结果为未定义

指令示例：

- Predicate: 控制计算过程中向量中对应位置是否参与运算

// 生成 (i, i + 1, i + 2, ..., i + vl - 1) < num 的 predicate

```
svbool_t pg = svwhilelt_b32(i, num)
```

- 创建predicate

// 生成一个全为 true 的 predicate

```
svbool_t pg = svptrue_b32()
```

- 内存数据存取

// 从地址 base 中加载连续向量

```
svld1(pg, *base)
```

// 从地址 base 中加载向量索引对应的数据

```
svld1_gather_index(pg, *base, vec_index)
```

指令性能（延迟/吞吐）原理及测试方法

- 延迟：2条依赖的指令之间的延迟
- 吞吐：流水意义下，1个cycle可以完成的指令数量

双精度FMLA延迟

```
void test_kernel(int64_t length)
{
    for (int x = 0; x < length; x += 1) {
        asm volatile (
            "fmmla v0.2d, v10.2d, v11.2d \n"
            "fmmla v0.2d, v10.2d, v11.2d \n"
            "fmmla v0.2d, v10.2d, v11.2d \n"
            "fmmla v0.2d, v10.2d, v11.2d \n"
            "fmmla v0.2d, v10.2d, v11.2d \n"
            "fmmla v0.2d, v10.2d, v11.2d \n"
            "fmmla v0.2d, v10.2d, v11.2d \n"
            "fmmla v0.2d, v10.2d, v11.2d \n"
            "fmmla v0.2d, v10.2d, v11.2d \n"
            "fmmla v0.2d, v10.2d, v11.2d \n"
            :
            : "v0", "v1", "v2", "v3"
            , "v4", "v5", "v6", "v7"
            , "v8", "v9", "v10", "v11"
            , "memory", "cc"
        );
    }
}
```

10条指令50个cycle，**双精度FMLA延迟为5**

双精度FMLA吞吐

```
void test_kernel(int64_t length)
{
    for (int x = 0; x < length; x += 1) {
        asm volatile (
            "fmmla v0.2d, v10.2d, v11.2d \n"
            "fmmla v1.2d, v10.2d, v11.2d \n"
            "fmmla v2.2d, v10.2d, v11.2d \n"
            "fmmla v3.2d, v10.2d, v11.2d \n"
            "fmmla v4.2d, v10.2d, v11.2d \n"
            "fmmla v5.2d, v10.2d, v11.2d \n"
            "fmmla v6.2d, v10.2d, v11.2d \n"
            "fmmla v7.2d, v10.2d, v11.2d \n"
            "fmmla v8.2d, v10.2d, v11.2d \n"
            "fmmla v9.2d, v10.2d, v11.2d \n"
            :
            : "v0", "v1", "v2", "v3"
            , "v4", "v5", "v6", "v7"
            , "v8", "v9", "v10", "v11"
            , "memory", "cc"
        );
    }
}
```

10条指令10个cycle，**双精度FMLA吞吐为1**

单精度FMLA延迟

```
void test_kernel(int64_t length)
{
    for (int x = 0; x < length; x += 1) {
        asm volatile (
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v0.4s, v10.4s, v11.4s \n"
            :
            : "v0", "v1", "v2", "v3"
            , "v4", "v5", "v6", "v7"
            , "v8", "v9", "v10", "v11"
            , "memory", "cc"
        );
    }
}
```

10条指令50个cycle，**单精度FMLA延迟为5**

单精度FMLA吞吐

```
void test_kernel(int64_t length)
{
    for (int x = 0; x < length; x += 1) {
        asm volatile (
            "fmmla v0.4s, v10.4s, v11.4s \n"
            "fmmla v1.4s, v10.4s, v11.4s \n"
            "fmmla v2.4s, v10.4s, v11.4s \n"
            "fmmla v3.4s, v10.4s, v11.4s \n"
            "fmmla v4.4s, v10.4s, v11.4s \n"
            "fmmla v5.4s, v10.4s, v11.4s \n"
            "fmmla v6.4s, v10.4s, v11.4s \n"
            "fmmla v7.4s, v10.4s, v11.4s \n"
            "fmmla v8.4s, v10.4s, v11.4s \n"
            "fmmla v9.4s, v10.4s, v11.4s \n"
            :
            : "v0", "v1", "v2", "v3"
            , "v4", "v5", "v6", "v7"
            , "v8", "v9", "v10", "v11"
            , "memory", "cc"
        );
    }
}
```

10条指令5个cycle，**单精度FMLA吞吐为2**

芯片性能计算方法

性能 = 核数 * 频率 * FMLA吞吐 * 向量化长度 * 2(加乘)

例1：鲲鹏920单核双精度计算性能

- 1核、2.6GHz、FMLA吞吐为1、128bit=2*double
- 性能 = 1 * 2.6GHz * 1 * 2 * 2 = 10.4 GFlops

例2：鲲鹏920单CPU单精度计算性能

- 64核、2.6GHz、FMLA吞吐为2、128bit=4*float
- 性能 = 64 * 2.6GHz * 2 * 4 * 2 = 2662.4 GFlops

例3：Intel Xeon 6248 单核双精度计算性能

- 1核、3.8GHz、FMLA吞吐为2、512bit=8*double
- 性能 = 1 * 3.8Ghz * 2 * 8 * 2 = 121.6 GFlops

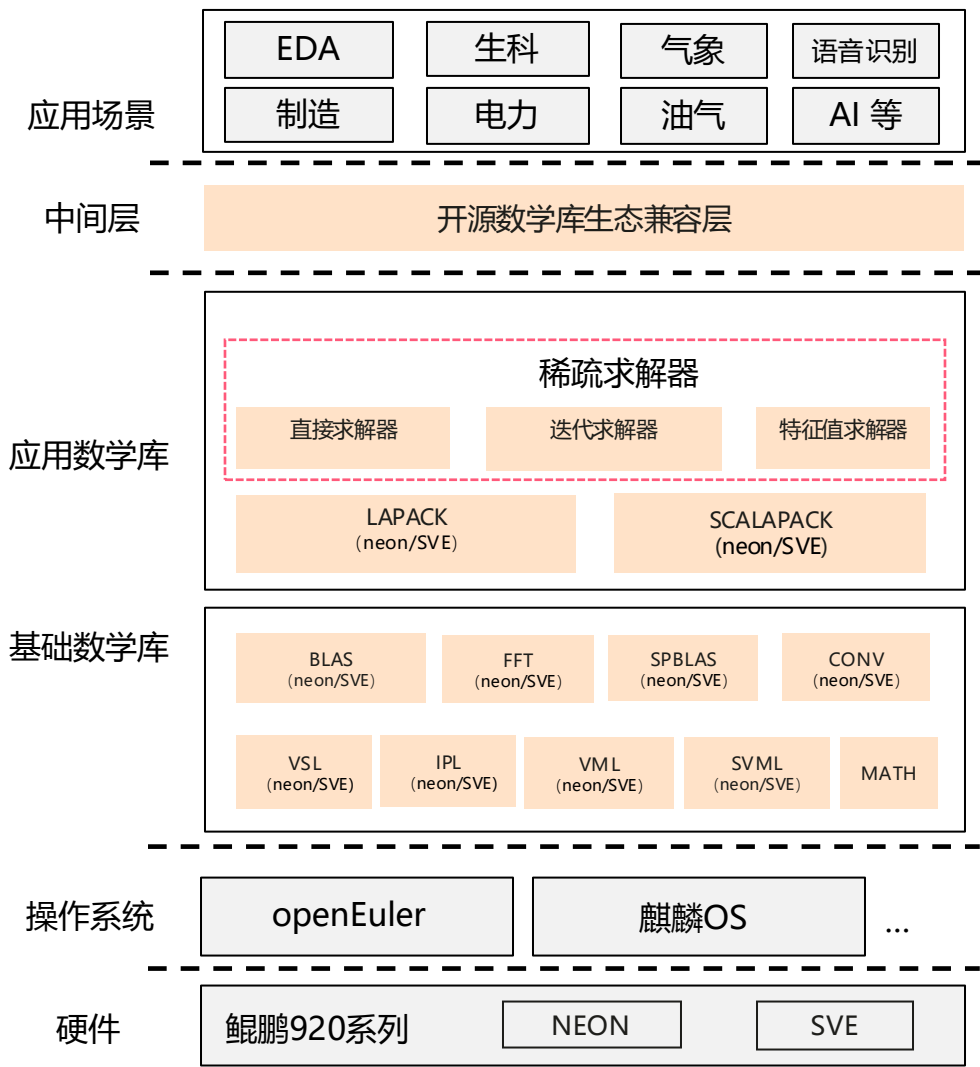
例4：Intel Xeon 6248 单CPU双精度计算性能

- 20核、2.5GHz、FMLA吞吐为2、512bit=8*double
- 性能 = 20 * 2.5GHz * 2 * 8 * 2 = 1600 GFlops

Mode	Base	Turbo Frequency/Active Cores												
		1	2	3	4		13	14	15	16	17	18	19	20
Normal	2,500MHz	3,900MHz	3,900MHz	3,700MHz	3,700MHz		3,400MHz	3,400MHz	3,400MHz	3,400MHz	3,200MHz	3,200MHz	3,200MHz	3,200MHz
AVX2	1,900MHz	3,800MHz	3,800MHz	3,600MHz	3,600MHz		3,000MHz	3,000MHz	3,000MHz	3,000MHz	2,800MHz	2,800MHz	2,800MHz	2,800MHz
AVX512	1,600MHz	3,800MHz	3,800MHz	3,600MHz	3,600MHz		2,700MHz	2,700MHz	2,700MHz	2,700MHz	2,500MHz	2,500MHz	2,500MHz	2,500MHz

数学库

鲲鹏数学库概述：多领域全功能数学库生态使能



鲲鹏数学库（Kunpeng Math Library，简称KML）提供了基于鲲鹏920系列优化的高性能数学函数，所有接口由C/C++、汇编语言实现，接口兼容Fortran语言调用。KML基于armv9架构，使能底层**NEON**、**SVE**指令加速，通过数据预取、编译优化、数据重排等手段对函数进行深度优化

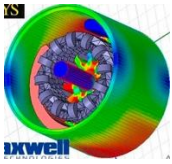
类型	数学库	功能说明	底层指令使用情况	使能领域
基础数学库	BLAS	基础线性代数库	NEON/SVE	教育、EDA、气象、电力、生科
	SPBLAS	稀疏基础线性代数库	NEON/SVE	EDA
	VML	向量函数库	NEON/SVE	EDA
	MATH	基础函数库	无	气象、制造
	FFT	Fourier变换库	NEON/SVE	EDA、教育、油气、生科
	IPL	插值计算库	NEON/SVE	EDA
	CONV	卷积计算库	NEON/SVE	EDA
应用数学库	Numpy	Python的数值计算扩展库	NEON/SVE	EDA
	LAPACK	线性代数库	NEON/SVE	制造、教育、EDA
	SCALAPACK	集群版线性代数库	NEON/SVE	制造、教育、EDA
	迭代求解器	稀疏矩阵迭代法求解+预条件子	NEON/SVE	制造、气象
	混合精度求解	大规模稀疏矩阵的混合精度算法	NEON/SVE	制造
	特征值求解器	大规模稀疏矩阵特征值求解	NEON/SVE	教育
	直接求解器	稀疏矩阵直接法求解	NEON/SVE	EDA、制造

线性代数基础介绍

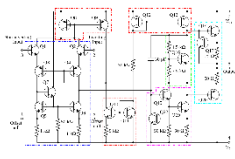
领域应用



化学与材料



电磁仿真



模拟电路



结构力学

微分方程

数学物理建模

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} = F(x)$$

泊松方程（椭圆方程）

$$\frac{\partial^2 u}{\partial t^2} = \alpha^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) + F(x, t)$$

波动方程（双曲方程）

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) + F(x, t)$$

热传导方程（抛物方程）

$$\frac{\partial(\rho u)}{\partial t} + \nabla \cdot (\rho u \otimes u) = -\nabla \cdot pI + \nabla \cdot \tau + \rho g$$

纳维-斯托克斯方程

线性化&数值离散

线性方程组求解

各种矩阵运算

多数HPC领域问题经过数学物理建模之后，就转换为线性代数问题，通过各种矩阵运算和线性方程组求解来得到最终的结果。

矩阵乘向量

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

$$c_i = a_{i1} * b_1 + a_{i2} * b_2 + a_{i3} * b_3$$

矩阵乘矩阵

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

$$c_{ij} = a_{i1} * b_{1j} + a_{i2} * b_{2j} + a_{i3} * b_{3j}$$

矩阵分解

当矩阵满足一定条件的情况下可以将矩阵分解成两个或多个矩阵相乘的形式

线性方程组求解： $Ax = b$ ，求解向量x

鲲鹏数学库：BLAS/LAPACK介绍

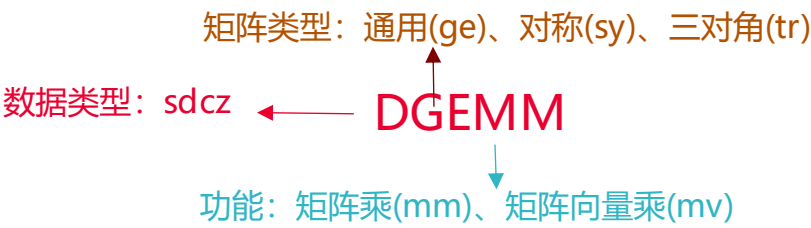
应用场景： BLAS和LAPACK是线性代数计算中广泛使用的两个重要库，专为高效执行线性代数操作而设计，两者共同为科学计算和工程应用提供了强大的线性代数支持，在流体力学(CFD)、结构分析、电磁仿真等领域都被广泛使用。

BLAS功能

- Level1: 向量-向量计算
- Level2: 向量-矩阵计算
- Level3: 矩阵-矩阵计算
- Extension: 额外的基础矩阵/向量计算

接口介绍

BLAS提供与业界主流的线性代数库一致的标准接口，如：OpenBLAS, Intel MKL等，可以直接替换库链接，无需修改应用



LAPACK功能

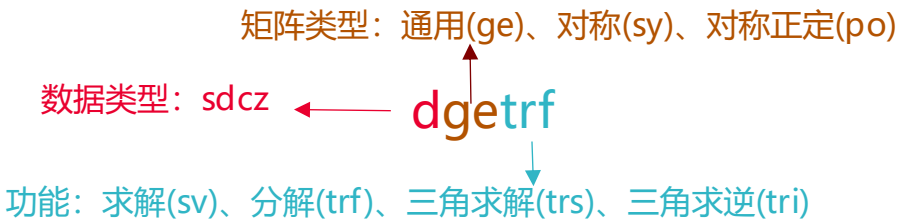
- (稠密) 线性方程组求解/分解【 $Ax=b$ 】
- 线性最小二乘法
- 标准/广义特征值与奇异值【 $Ax=\lambda x$ 】
- 正交分解：QR/RQ/LQ/QL分解
- 矩阵求逆

使用方式和接口分类

- 求解：矩阵分解为三角矩阵->三角矩阵求解
- 求逆：矩阵分解为三角矩阵->三角矩阵求逆
- 常见分解：LU(通用)/Cholesky(对称正定)/LDLT(对称)/QR类分解
- 接口分类：driver直接计算(原始矩阵->求解结果)、computing中间过程(分解、三角求解)、辅助接口(范数计算、行交换等)

接口介绍

LAPACK提供与业界主流的线性代数库一致的标准接口，如：Netlib LAPACK, OpenBLAS, Intel MKL等，可以直接替换库链接，无需修改应用



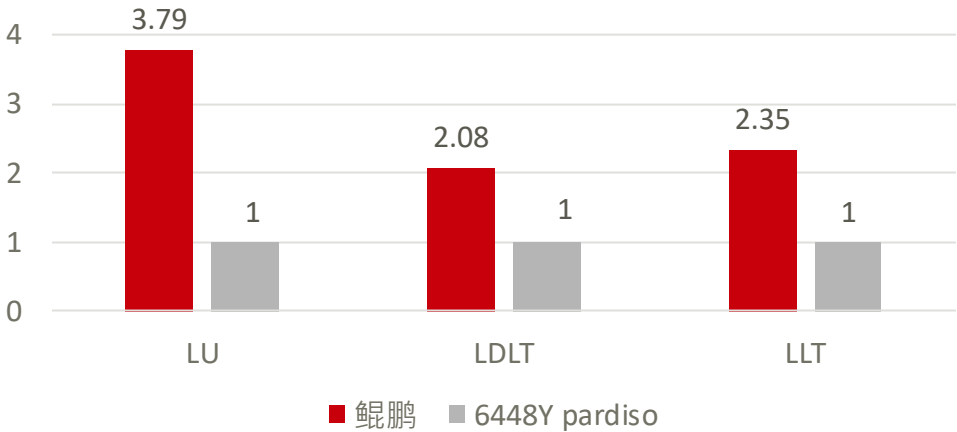
鲲鹏数学库：直接稀疏求解器(天元)介绍

应用场景：许多科学和工程问题可以由(偏)微分方程表述。(偏)微分方程的数值解可以转化成稀疏线性方程组求解问题。直接稀疏求解器由于其稳定性高，可以处理较病态的矩阵，所以广泛应用于EDA的电磁仿真和电路仿真场景。

直接求解器功能：求解稀疏方程组 $AX=b$

- 并行方式：单进程多线程、多进程多线程，可灵活设置线程数
- 稀疏矩阵压缩格式矩：**CSR**, COO, CSC
- 矩阵重排算法：重排算法可决定给后续流程的内存占用和性能，自研的KRDR 矩阵重排算法，相对于开源重排算法可加速后续流程50%，并且能保证多次运行结果稳定
- 分解方式：对称正定矩阵->Cholesky 分解，对称矩阵->LDLT分解，一般矩阵->LU 分解

直接求解器性能相比Intel Pardiso平均提升**174%**
矩阵求解性能对比



直接求解器使用方式：

直接求解器提供了创建矩阵接口，初始化求解器、分析、分解以及求解全阶段接口，并支持各阶段设置可选属性

初始化求解器

Init

分析

包括Matching, 重排以及符号分解

Analyze

数值分解

将原始矩阵分解为三角矩阵

Factorize

求解

三角矩阵求解

Solve

销毁求解器

Clean

① 矩阵非零元分布不变，仅元素值变化时，无需重新分析

② 矩阵不变，仅右端项改变时，无需重新分析和分解



获取鲲鹏数学库及相关信息渠道

类别	交付件	链接	使用说明
数学库	源码	https://www.hikunpeng.com/developer/boostkit/library https://www.hikunpeng.com/developer/hpc/hpckit-download	• 开源适配层可在鲲鹏社区下载源码并编译使用
	软件包		• 当前支持rpm包安装与HPCKit工具安装
	开发文档		• 包含开发指南和迁移文档

MPI

MPI介绍

Message Passing Interface，消息通信接口规范。

概念：MPI标准定义的一组具有可移植性的并行技术编程接口，以及相关协议和语义规范。

场景：并行计算，通过在不同进程间共享及同步数据（计算数据，状态数据），使计算集群高效完成共同的计算任务。

特点：高性能、大规模、可伸缩、可移植。

- 两种典型的点对点操作

双边操作：MPI_Send/MPI_Recv

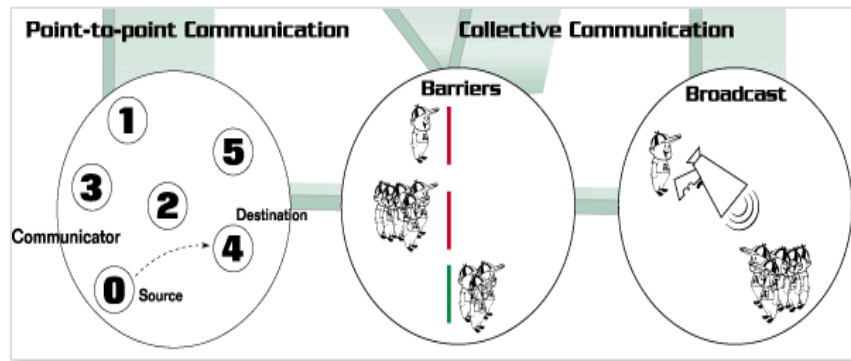
单边操作：MPI_Put/MPI_Get

- 三种类型集合通信操作：

一对多（One to Many）：MPI_Bcast、MPI_Scatter

多对一（Many to One）：MPI_Reduce、MPI_Gather

多对多（Many to Many）：MPI_Allreduce、MPI_Allgather、MPI_Alltoall



标准

MPI 1.0

MPI 1.1

MPI 1.2

MPI 1.3

MPI 2.0

MPI 2.1

MPI 2.2

MPI 3.0

MPI 3.1

MPI 4.0

开源实现

OpenMPI

MPICH

MVAPICH

商业实现

Nvidia HPCX

IBM Spectrum MPI

Hyper MPI

Intel IMPI

Cray IMPI

现通用标准为MPI 3.1，最新标准为MPI 4.0。

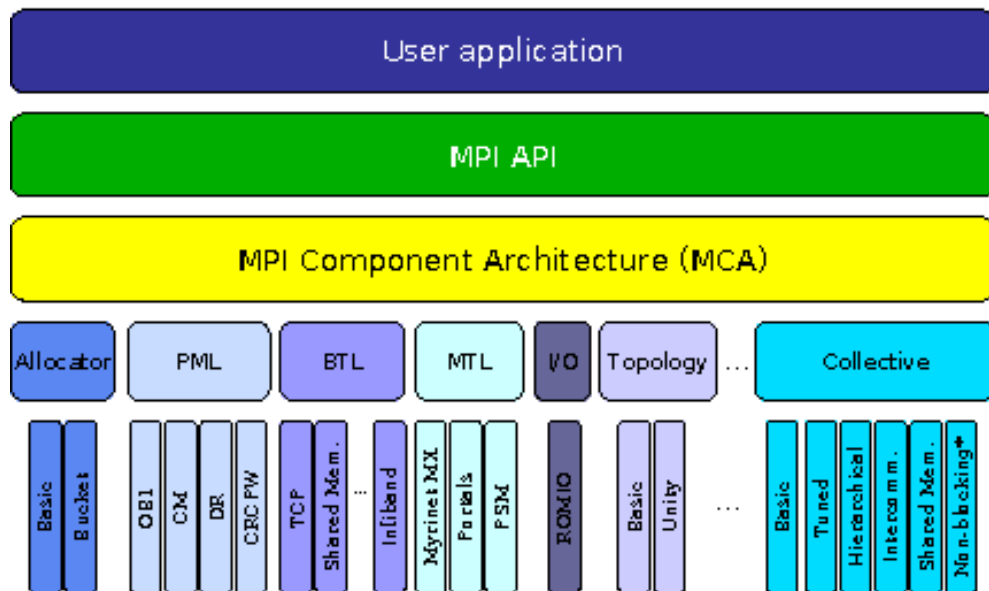
目前业界MPI基本为开源或者免费。

OpenMPI简介

Open MPI是著名的开源MPI软件实现之一（另一个是MPICH）。



Open MPI:
Open Source High Performance Computing



亮点1:

Open MPI全功能模块基于插件化的框架开发，有利于多个团队协作开发。如HPC-X就是在collective框架下添加了hcoll组件，对接了Mellanox的HCOLL通信库；HMPI也是在collective框架下添加了ucg组件，对接了自研的HUCG。

亮点2:

Open MPI社区活跃，多个组织、厂商参与贡献。如CISCO、IBM、NVIDIA、FUJITSU等等。社区的版本稳定演进，目前演进到5.x版本，积极推进MPI-4标准实现。

一句话总结:

OpenMPI开源社区活跃，整体软件架构模块化、插件化，利于二次开发。同时在各大超算也有实践经验。

Open MPI State of the Union XIV Community Meeting SC21

Jeff Squyres

George Bosilca

Joshua Hursey
Geoff Paulsen
Austen Lauria

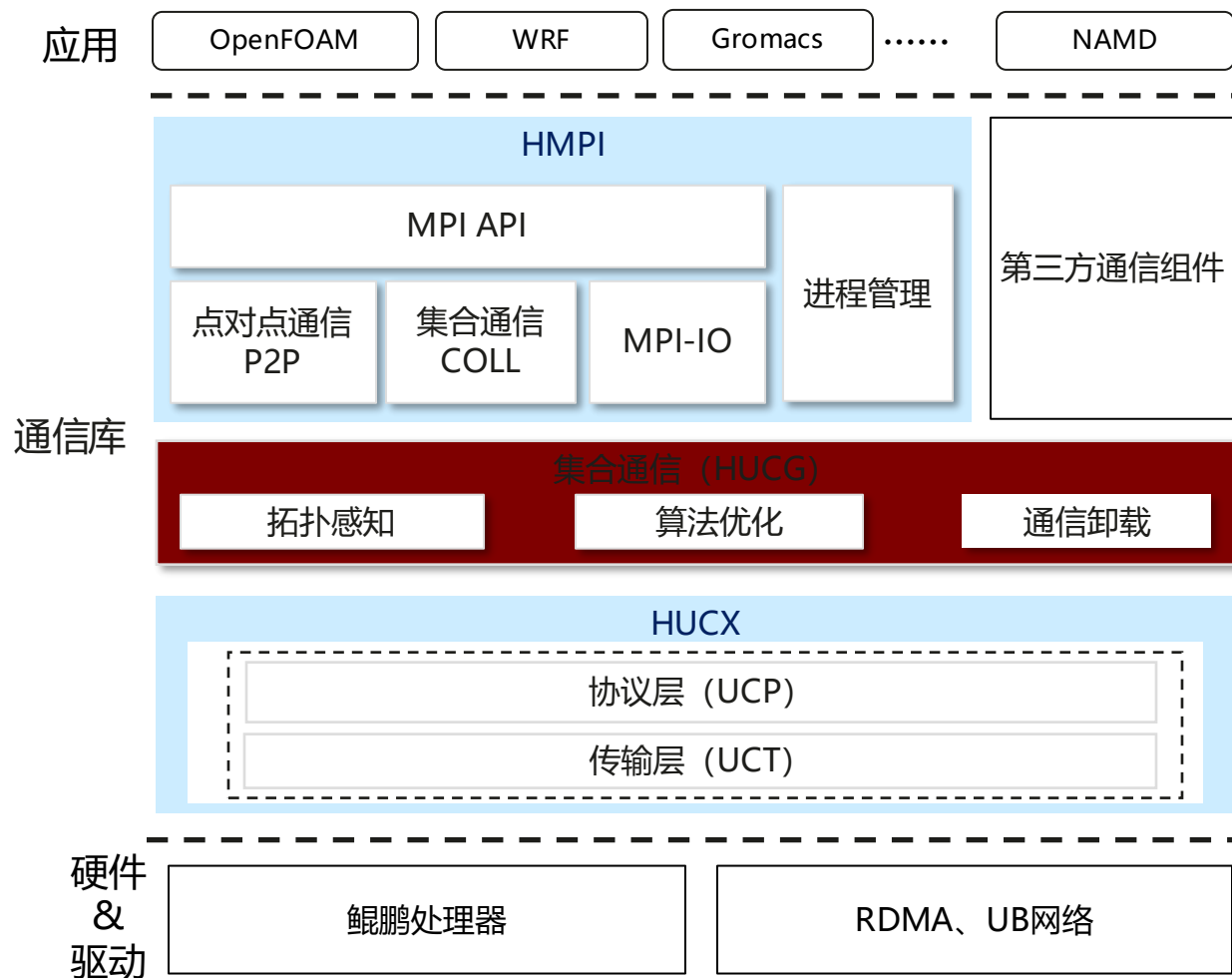
Tomislav Janjusic

Howard Pritchard

Takafumi Nose



Hyper MPI概述



概述

Hyper MPI基于OpenMPI，遵循MPI 3.1标准，配合通信算法与协议优化，软硬协同，实现拓扑感知通信自动优化。

关键特性

- 通过多RDMA网口负载均衡以及拓扑感知的通信算法等关键技术，实现通信性能提升。针对大包问题做了针对性优化。
- 面向E级算力大系统，HMPI通过对启动流程优化，实现大规模进程快速启动。

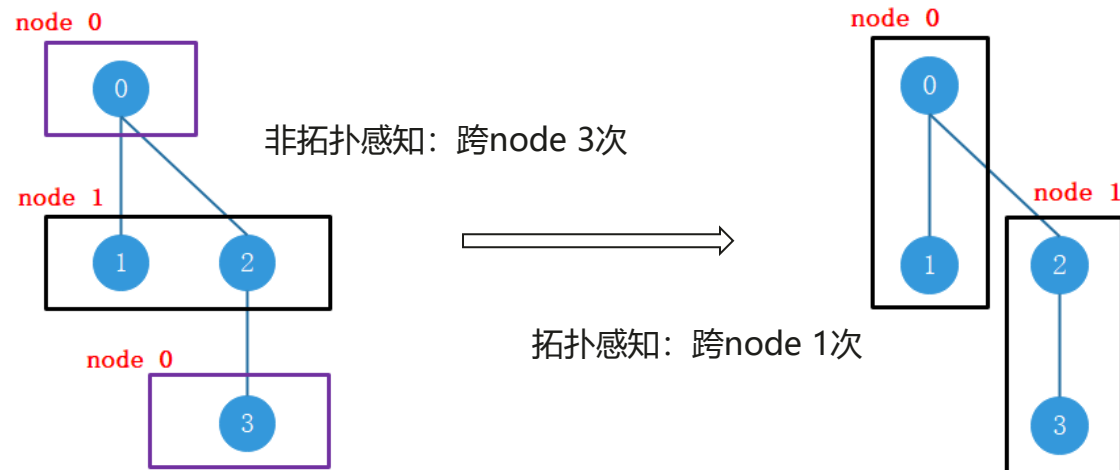
版本发布

- Hyper MPI发布版
<https://www.hikunpeng.com/developer/hpc/hypermapi>
- Hyper MPI开源版社区
<https://gitee.com/kunpengcompute/xucg/tree/huawei>
<https://gitee.com/kunpengcompute/hucx/tree/huawei/>
<https://gitee.com/kunpengcompute/hmpi/tree/huawei/>

Hyper MPI关键特性——拓扑感知算法

Memory Hierarchy	Latency
Node	1.8us
Socket	0.5us
NUMA	0.3us

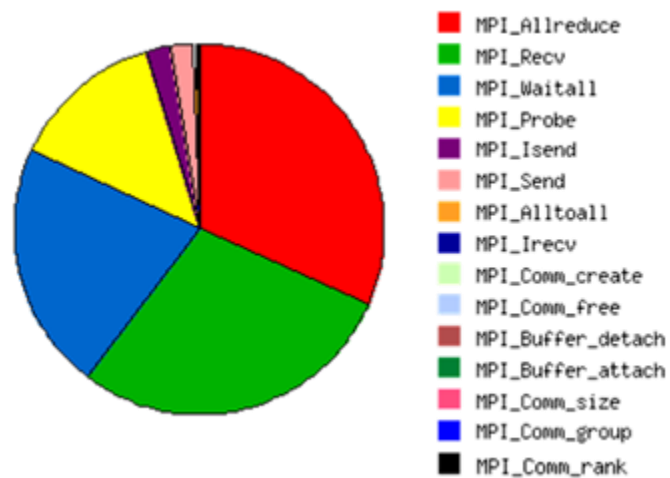
鲲鹏920 不同层级P2P时延



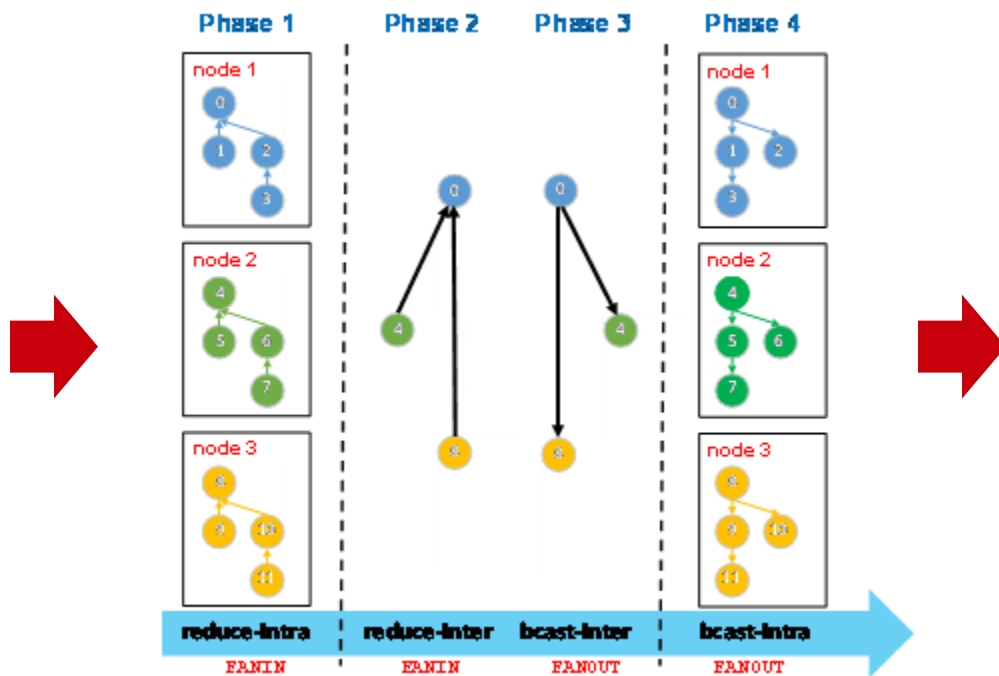
OpenFOAM优化案例

- **OpenFOAM**: 开源的CFD软件
- **性能分析**: 可以发现小包MPI_Allreduce占比非常高, 这是由于其求解器核心是共轭梯度法 (**Conjugate Gradient**)
- **优化方案**: 拓扑感知的K-nomial tree算法, 小包allreduce接口性能提升 **40%**, 客户用例端到端性能提升**7%**

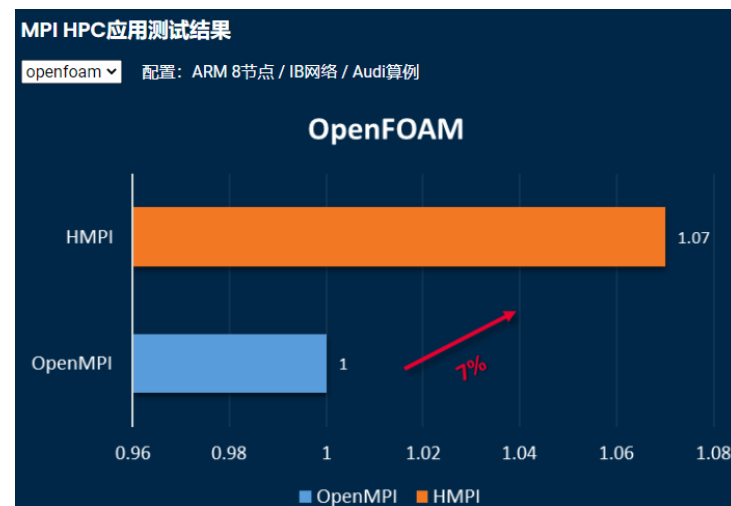
OpenFoam通信占比



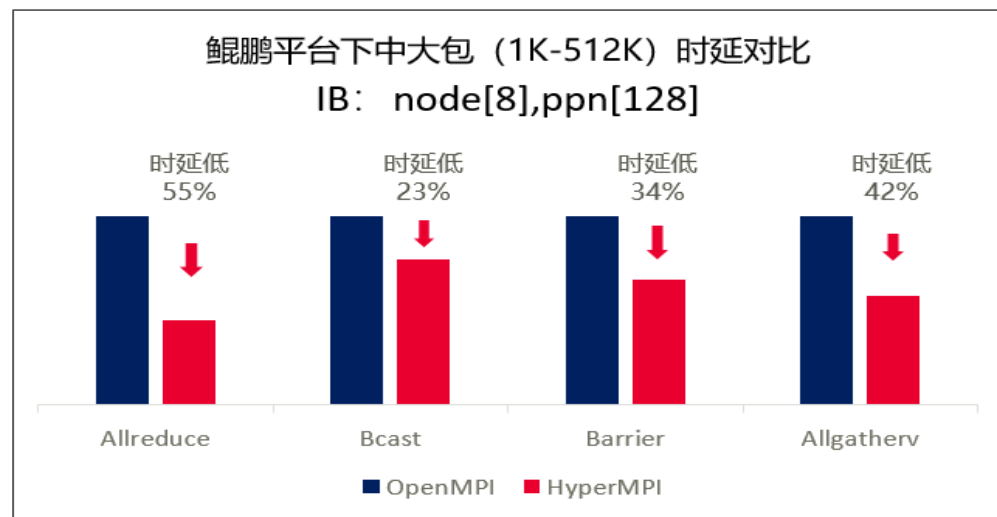
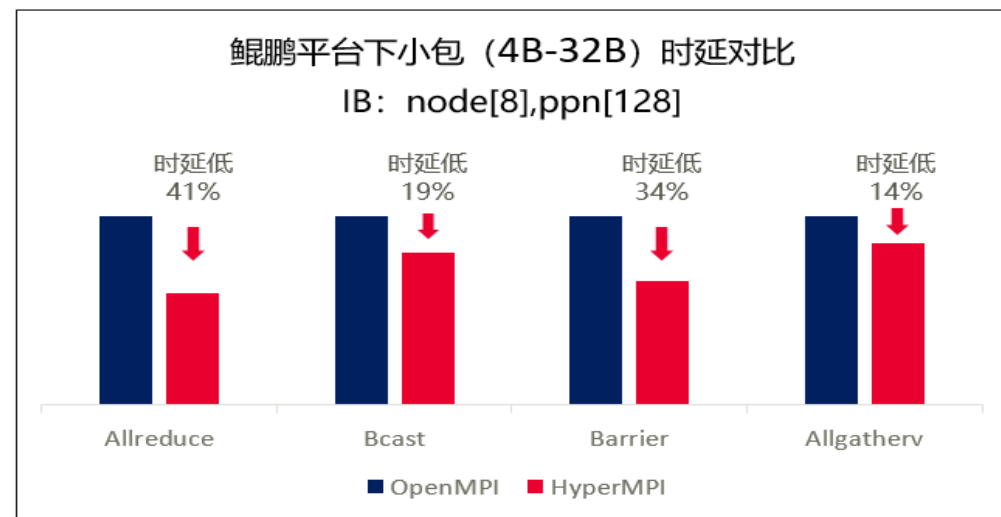
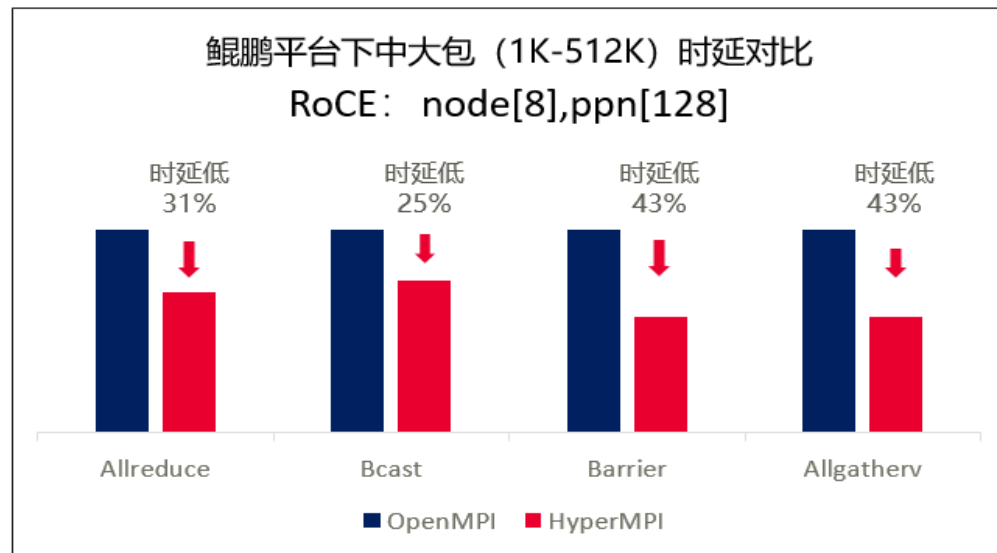
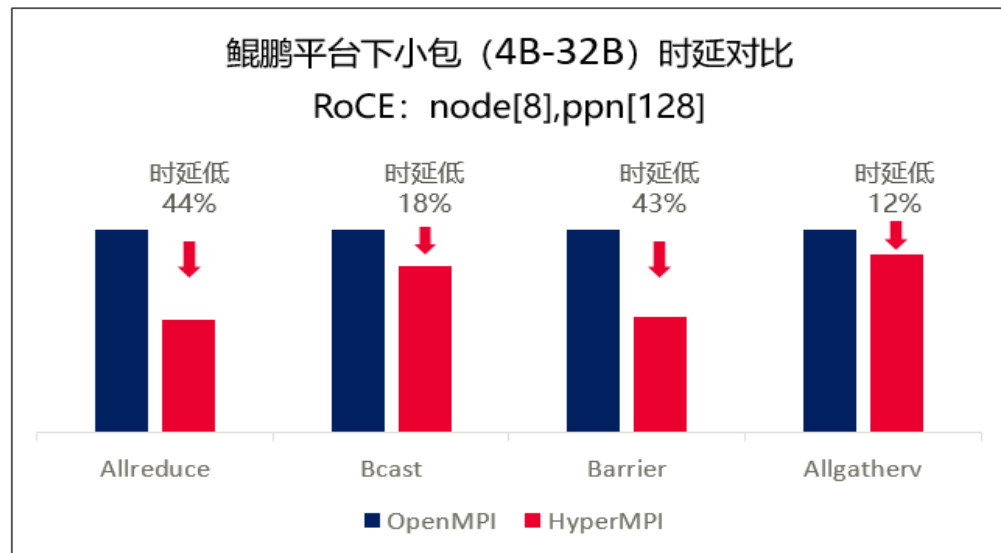
Topo-aware K-nomial tree算法



鲲鹏8节点, OpenFoam客户算例



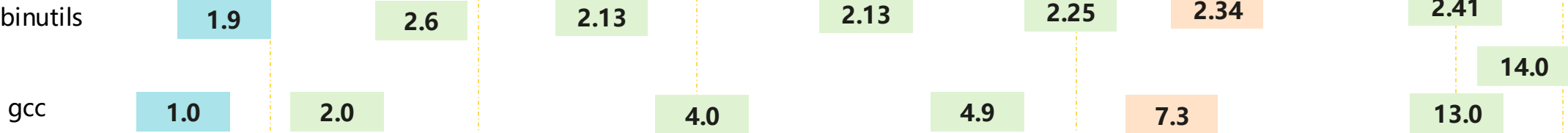
Hyper MPI集合通信性能对比



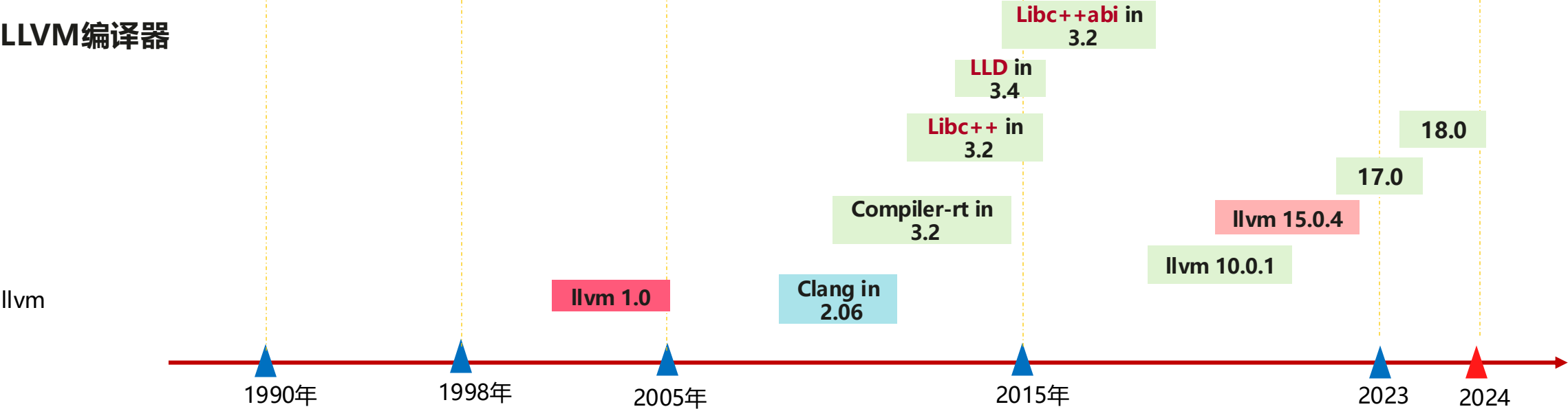
编译器

开源编译器演进

GCC编译器



LLVM编译器

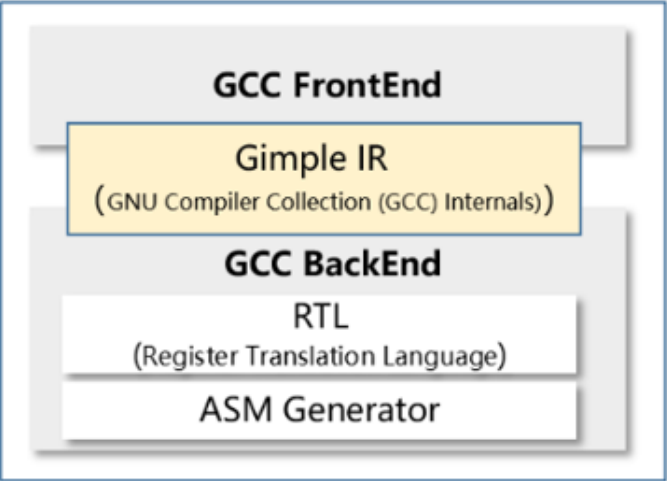


商业编译器正逐步往LLVM发展，LLVM正成为未来编译器主流

开源编译器

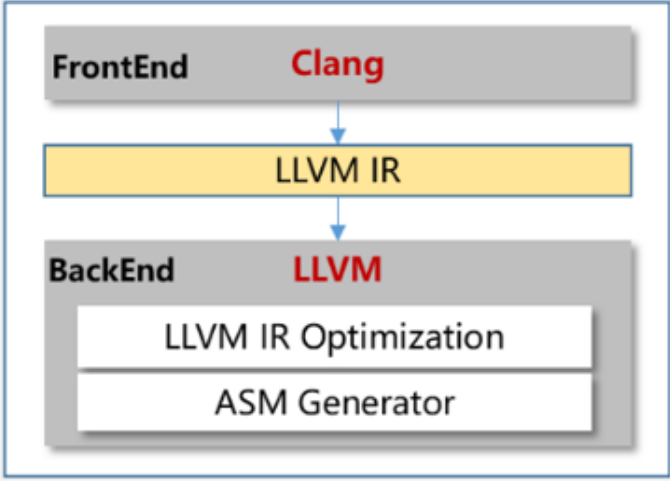
GCC编译器

前端、IR、后端编程界面不清晰，耦合较重，难以独立发展；License不友好



LLVM编译器

LLVM IR有简洁的定义，清晰的编程接口，前端和后端可以独立发展，容易演进；License友好

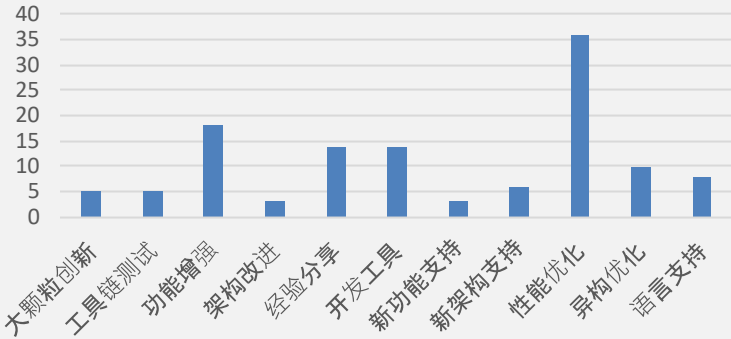


LLVM社区活跃度、生态发展、新技术演进已经远超过GCC

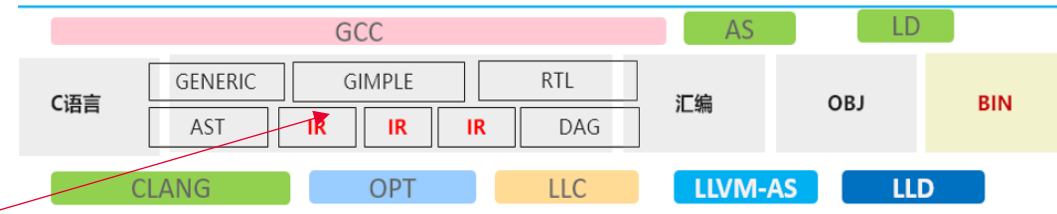
商业公司编译器

公司	编译器策略和趋势
Apple	全面转向LLVM。MacOS上的C/C++/Obj-C编译器使用LLVM，新编程语言页使用LLVM
Google	Android偏底层C/C++ NDK工具链使用LLVM，移除GCC
ARM	AHC（ARM High Performance Computing）编译器基于LLVM
NVIDIA	OpenCL编译器基于LLVM，NVCC编译器也转型LLVM

传统芯片和科技大厂在近3年LLVM开发者大会上纷纷发表自身在LLVM的研究，发表数量远超GCC峰会



编译器优化技术： 自动向量化



- 在并行计算中，自动矢量化是自动并行化的一个特殊场景，其中计算机程序从一次处理一对操作数的标量实现转换为矢量实现，它能在一次操作中同时处理多对操作数。比如，基于AArch64的鲲鹏920处理器，具有32个128 bits的矢量寄存器，可以一次操作4路32位或者2路64位的数据。

✓ 循环向量化原理：

```
void foo(int *a, int *b,
        int *c, int n) {
    for (i=0; i<n; i++)
        a[i] = b[i] + c[i];
}
```



```
void foo(int *a, int *b,
        int *c, int n) {
    for (i=0; i < n/4; i+=4) {
        a[i+0] = b[i+0] + c[i+0];
        a[i+1] = b[i+1] + c[i+1];
        a[i+2] = b[i+2] + c[i+2];
        a[i+3] = b[i+3] + c[i+3];
    }
}
```



```
void foo(int *a, int *b,
        int *c, int n) {
    for (i=0; i < n/4; i+=4)
        a[0..3] = b[0..3] + c[0..3];
}
```

✓ SLP向量化原理：
(superword-level parallelism)

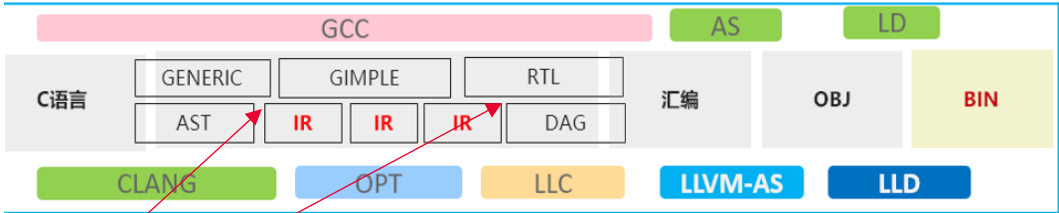
```
void foo(int a1, int a2,
        int b1, int b2,
        int *A) {
    A[0] = a1*(a1 + b1);
    A[1] = a2*(a2 + b2);
    A[2] = a1*(a1 + b1);
    A[3] = a2*(a2 + b2);
}
```



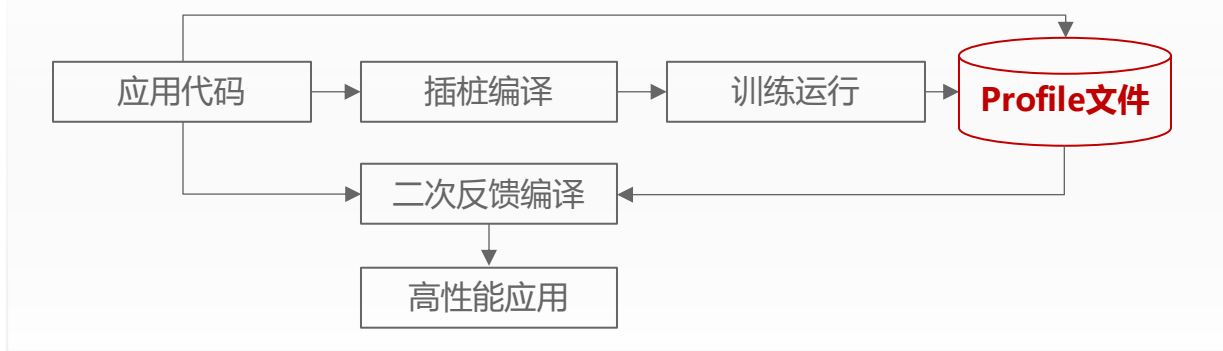
```
void foo(int a1, int a2,
        int b1, int b2,
        int *A) {
    A[0..3] = {a1,a2,a1,a2} * ({a1,a2,a1,a2} + {b1,b2,b1,b2});
}
```

编译器优化技术：反馈优化

数据库类应用，如MySQL其代码特点是控制流较多、对数据分段访问较多，此类应用非常适合编译器的反馈类优化技术，如PGO，AutoFDO，BOLT等，毕昇编译器也对此类技术进行了支持。



反馈类优化流程图



根据程序的profile信息，可以做以下优化：

- (1) 代码块重排：根据分支执行概率，做代码块重排；
- (2) 函数inline：根据函数执行冷热程度，指导inline策略；
- (3) 冷热分区：根据函数执行冷热程度，链接器进行函数聚合。
- (4) 辅助寄存器分配：根据代码块执行冷热程度，调整寄存器分配策略；
- (5)

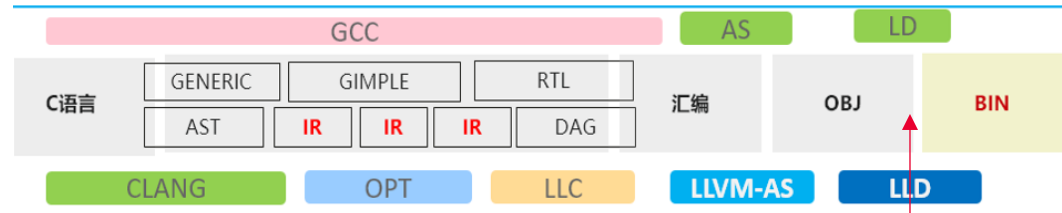
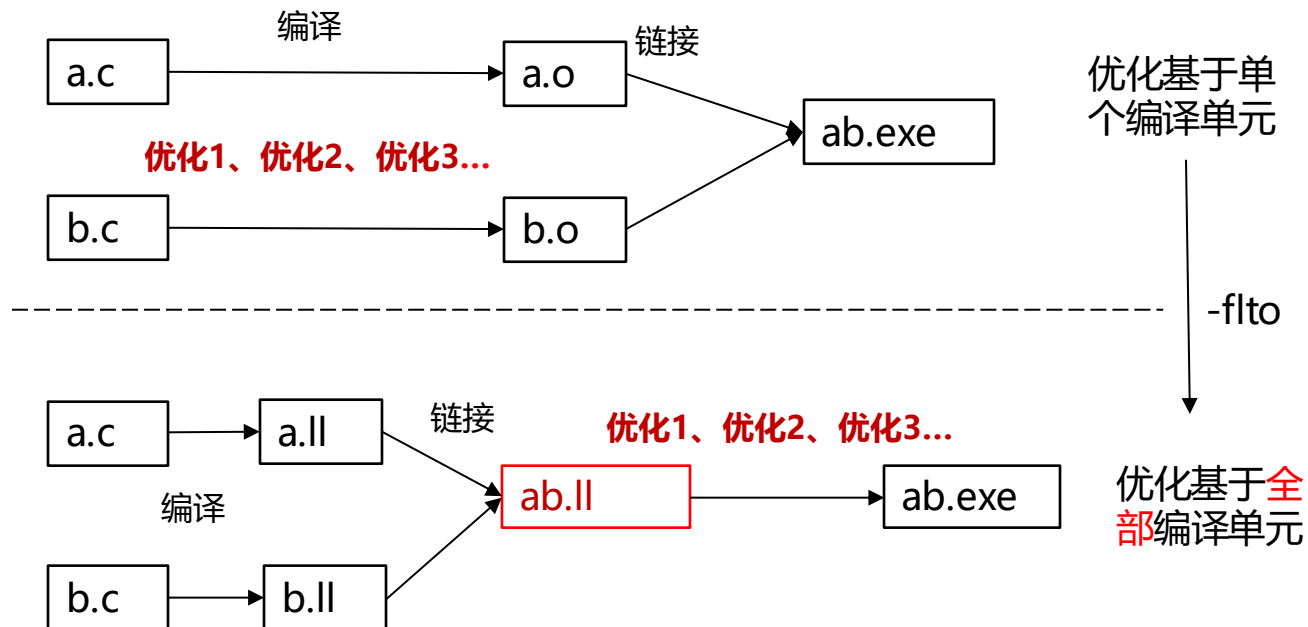
应用效果

MySQL 8.0.20应用tpcc较GCC 9.3.1 **性能提升8~15%**
随着毕昇编译器对反馈优化技术的持续优化，应用效果可以进一步提升。

优化技术	简介
PGO	一般通过 软件插桩 的方法收集程序运行的profile数据，指导二次编译时做精准的、激进的优化
AutoFDO	一般通过 利用硬件 收集程序运行的profile数据，指导二次编译时做精准的、激进的优化
BOLT	在 二进制层面 利用硬件的profile信息，进行精准的、激进的优化

编译器优化技术：连接时优化

链接时优化 (LTO)原理：跨文件、过程间优化



LTO效果

- + 跨文件函数内联, 减小调用开销
- + 跨文件函数特化、常量传播, 消除冗余代码
- + 可以跨语言优化
- 编译时间更长
- 需要修改Makefile

LTO是成熟的编译优化技术、业界已经广泛使用

- 谷歌Android kernel使用LTO编译
- Linux kernel 5.12版本引入LTO支持
- SPECCPU, GaussDB等

KUPL

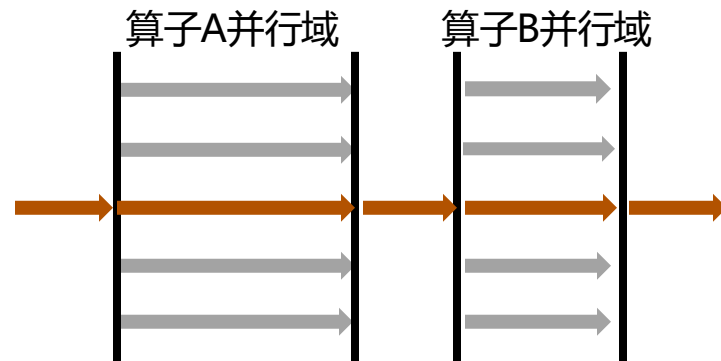
鲲鹏硬件编程框架与应用适配的挑战

挑战一：针对鲲鹏硬件的众核架构等特性，**传统多线程编程框架表达能力不足**，无法发挥硬件性能

- 现有编程模型框架局限性：OpenMP为例（HPC领域最广泛使用模型）

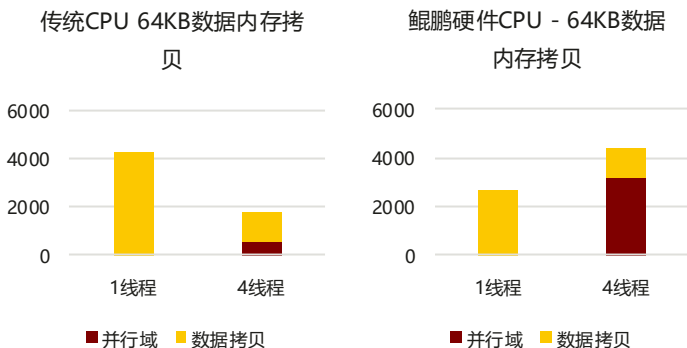
```
// OMP version
#pragma omp parallel for num_threads(36)
for (int i = 0; i < n; i++) {
    compute_kernel_A(i);
}
#pragma omp parallel for num_threads(36)
for (int i = 0; i < n; i++) {
    compute_kernel_B(i);
}
```

目标代码



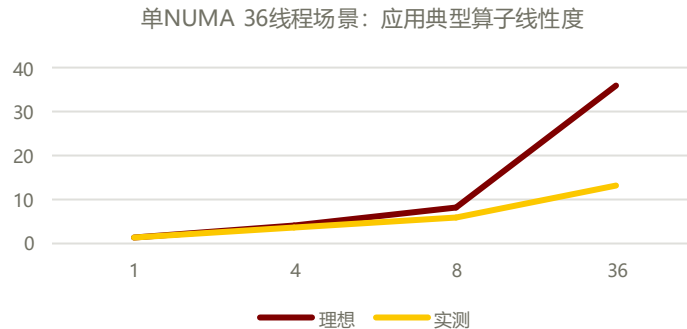
运行时执行逻辑

问题一：Fork-Join模式下，众核架构带来的一系列因素导致**OpenMP并行域启停**开销大幅放大



并行模式引起的性能问题

问题二：单一算子的并行，**线性度不理想**（数据规模小，频发的核间共享访问）导致的性能劣化



单算子并行引起的性能问题

鲲鹏硬件编程框架与应用适配的挑战

挑战二：多领域应用，多种类编程框架的统一管理与优化,缺失统一运行时底座

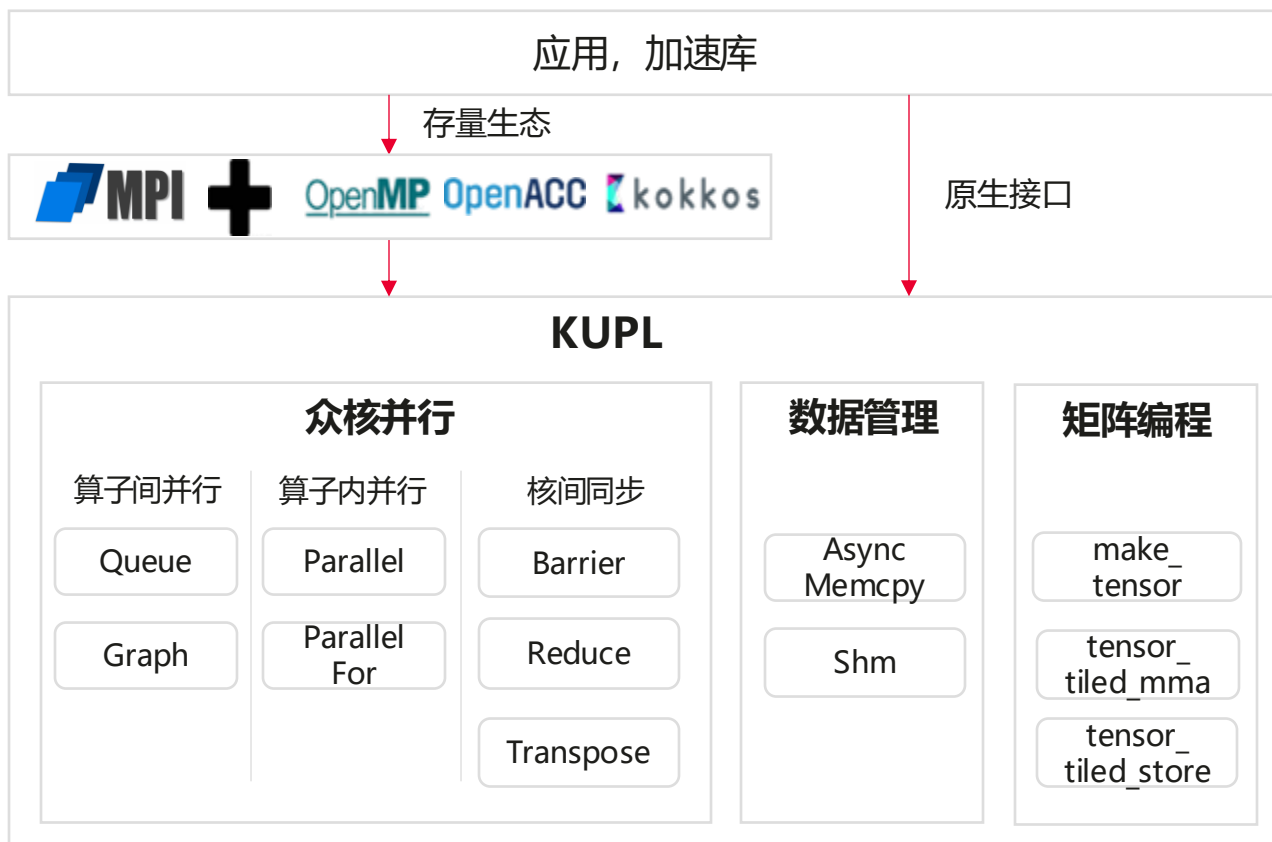
- 应用领域复杂化：传统HPC应用，AI4S应用
- 编程框架多样性：OpenMP, OpenACC, Kokkos, SYCL, Pytorch...



KUPL – Kunpeng Unified Parallel Library: 发挥硬件特性，提高编程易用性，兼容OpenACC等现有生态，构筑传统HPC&AI4S算子鲲鹏生态

KUPL软件架构

鲲鹏原生开发框架，对标Intel Level Zero和NV CUDA，北向兼容OpenMP、OpenACC等标准编程，发挥鲲鹏硬件和众核能力



1. 提供二类API

• 众核并行:

- 算子间并行，类似CUDA Stream/Event, Graph。
- 算子内并行，Parallel/Parallel_for
- 核间同步：Barrier, Reduce, Transpose。

- **数据管理：**异步内存拷贝对标cudaMemcpyAsync，实现数据移动与计算重叠；Shm对标CUDA IPC，加速NUMA间数据移动。

2. 与编译器配合，支持OpenMP，OpenACC等标准编程接口。

特性	CUDA	OpenMP	KUPL
算子内并行	Grid, Block	Parallel for	Parallel_for
核间同步	Barrier, Reduce	Barrier, Reduce	Barrier, Reduce
流/队列	Stream/Event	/	Queue/Event
动态图	/	Task	Task
静态图	Graph	/	Graph
内存分配	Memory Mgmt.	/	/
数据移动	Async memcpy	/	Async memcpy
内存通信	IPC	/	Shm
矩阵计算	wmma	/	mma

一：众核并行---计算图编程

基本概念

静态图即用于描述一个特定复杂问题的依赖关系图（DAG），将其中的计算、数据移动等任务定义成静态图中的点，将各个任务之间的依赖关系定义成静态图中的边。

KUPL提供让用户定义静态图的能力，并通过将静态图提交到动态图中执行实现多线程的并行执行与负载均衡调度。

推荐场景

符合以下特性的问题均可采用计算图进行解决

- 问题可拆分成多个任务且存在一定依赖关系
- 问题需要被反复提交执行，如HPC应用中的迭代计算等

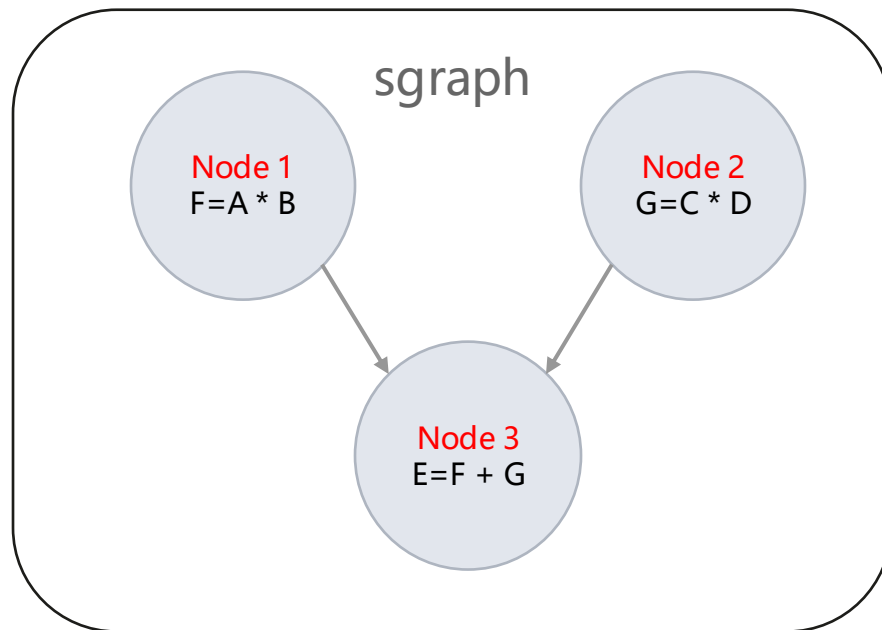
场景示例

以计算表达式 $E = A * B + C * D$ 为例进行说明：

针对上述问题可以拆分如下计算步骤：

1. 计算 $F = A * B$ 和 $G = C * D$
2. 计算 $E = F + G$

上述计算步骤可构建成为右图所示的静态图



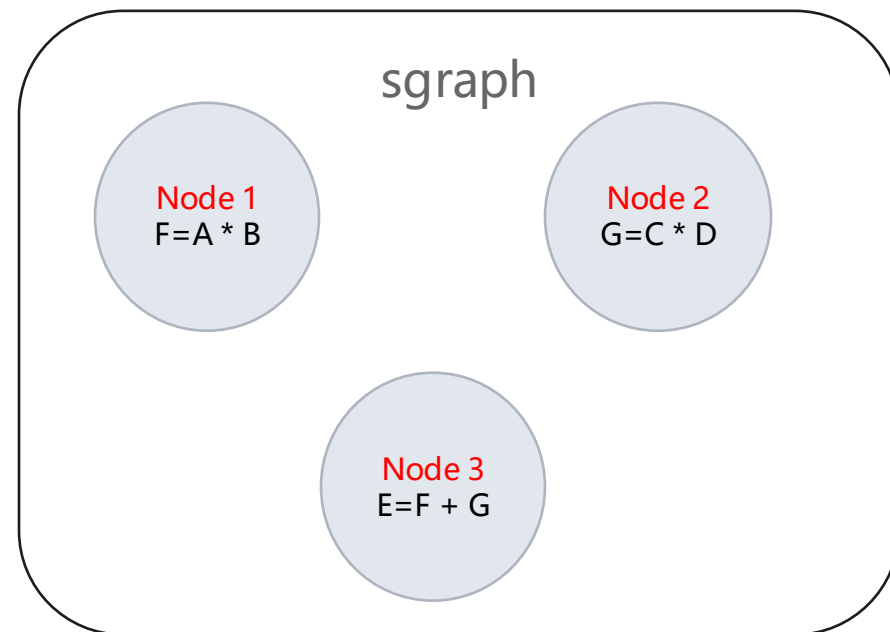
一：众核并行---计算图编程

● 计算表达式 $E = A * B + C * D$ 编程示例

```
auto sgraph = kupl_sgraph_create();

// Step1: 往静态图sgraph中添加计算节点
typedef struct data_axb {
    int *F_data; int *A_data; int *B_data;
} data_axb_t;
static void fun_axb(void *args) {
    data_axb_t *data = (data_axb_t *)args;           // 对入参进行解包操作
    int *F = data->F_data, *A = data->A_data, *B = data->B_data;
    (*F) = (*A) * (*B);
}
data_axb_t data1 = {
    .F_data = &F, .A_data = &A, .B_data = &B
};
kupl_sgraph_node_desc_t node1_desc = {
    .func = fun_axb,                                // 函数指针赋值
    .args = &data1                                  // 对入参进行打包操作
};
auto node1 = kupl_sgraph_add_node(sgraph, &node1_desc);

... // 同上配置node2_desc
auto node2 = kupl_sgraph_add_node(sgraph, &node2_desc);
... // 同上配置node3_desc
auto node3 = kupl_sgraph_add_node(sgraph, &node3_desc);
```



配置静态图节点的desc描述，从而调用
kupl_sgraph_add_node接口往静态图graph
中添加节点

一：众核并行---计算图编程

● 计算表达式 $E = A * B + C * D$ 编程示例

// Setp2: 添加静态图节点间依赖关系

`kupl_sgraph_add_dep`(node1, node3);

`kupl_sgraph_add_dep`(node2, node3);

// Step3: 将静态图封装成静态图任务提交至动态图执行, 并等待任务执行完毕

`auto graph = kupl_graph_create`(KUPL_ALL_EXECUTORS);

`kupl_sgraph_desc_t sgraph_desc = {`

`.sgraph = sgraph`

`};`

`kupl_task_info_t sgraph_info = {`

`.type = KUPL_TASK_TYPE_SGRAPH,`

`.desc = sgraph_desc`

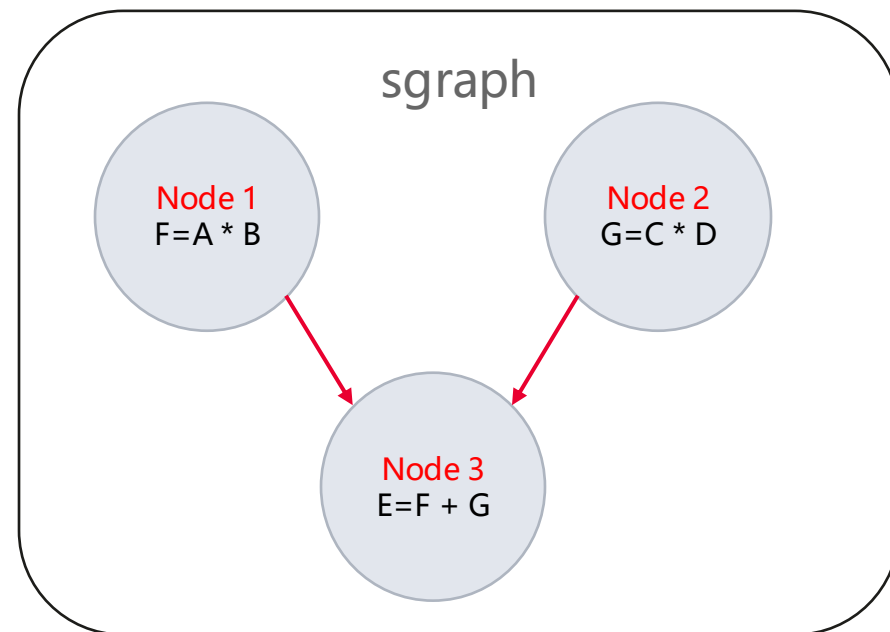
`};`

`kupl_graph_submit`(graph, &sgraph_info);

`kupl_graph_wait`(graph);

`kupl_graph_destroy`(graph);

`kupl_sgraph_destroy`(sgraph);



调用`kupl_sgraph_add_dep`接口定义静态图各节点的依赖

一：众核并行---计算图编程

● 计算图相关函数接口

描述	接口名称
创建静态图	kupl_sgraph_create
销毁静态图	kupl_sgraph_destroy
向静态图中添加任务节点	kupl_sgraph_add_node
向静态图中添加任务依赖	kupl_sgraph_add_dep
创建动态图	kupl_graph_create
销毁动态图	kupl_graph_destroy
向动态图中提交静态图任务	kupl_graph_submit
阻塞等待动态图中任务执行完毕	kupl_graph_wait

计算图相关函数接口鲲鹏社区链接：

https://www.hikunpeng.com/document/detail/zh/kunpenghpcs/hpckit/devg/KunoengHPCKit_developer_031.html

计算图代码Sample链接：

<https://gitee.com/openeuler/kupl-sample>

二：众核并行---多队列编程

基本概念

多队列编程也是使用依赖进行编程的方式。队列用于组织和执行一系列操作，这些操作按顺序在流中执行，但不同流之间的操作可以并行执行（如果硬件资源允许）。事件是一个轻量级的同步标记，用于记录时间点或标记流中的某个位置。事件可以插入到流中，用于事件同步或者队列间同步。

KUPL提供让用户定义队列、事件的能力，并通过将算子提交到队列上执行实现多线程的并行执行与依赖关系定义。

推荐场景

- 问题可拆分成多个任务且存在一定依赖关系
- 已有的 cuda、sycl 程序迁移至 cpu 上运行场景

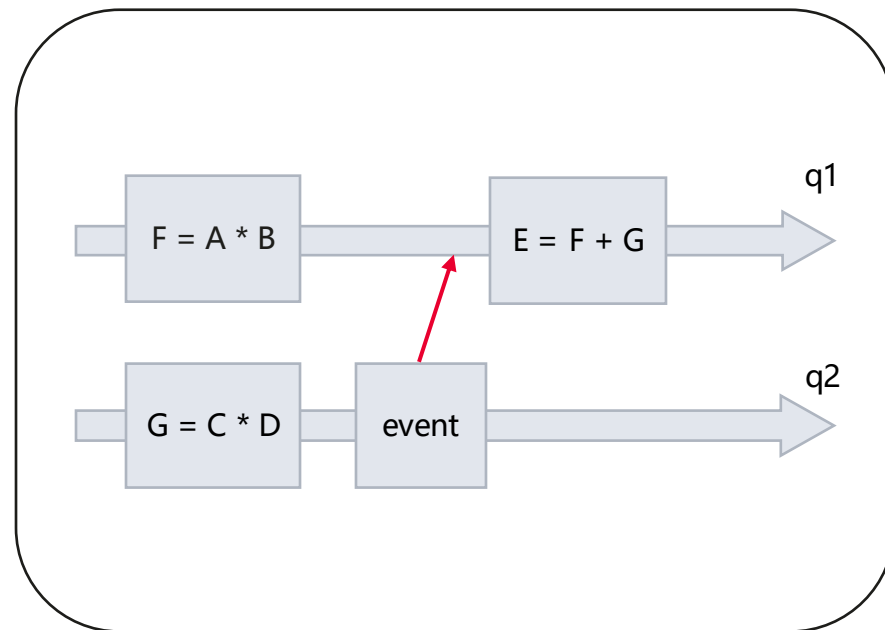
场景示例

同样以计算表达式 $E = A * B + C * D$ 为例进行说明：

针对上述问题可以拆分如下计算步骤：

1. 计算 $F = A * B$ 和 $G = C * D$
2. 计算 $E = F + G$

上述计算步骤可构建成为右图所示的队列和事件

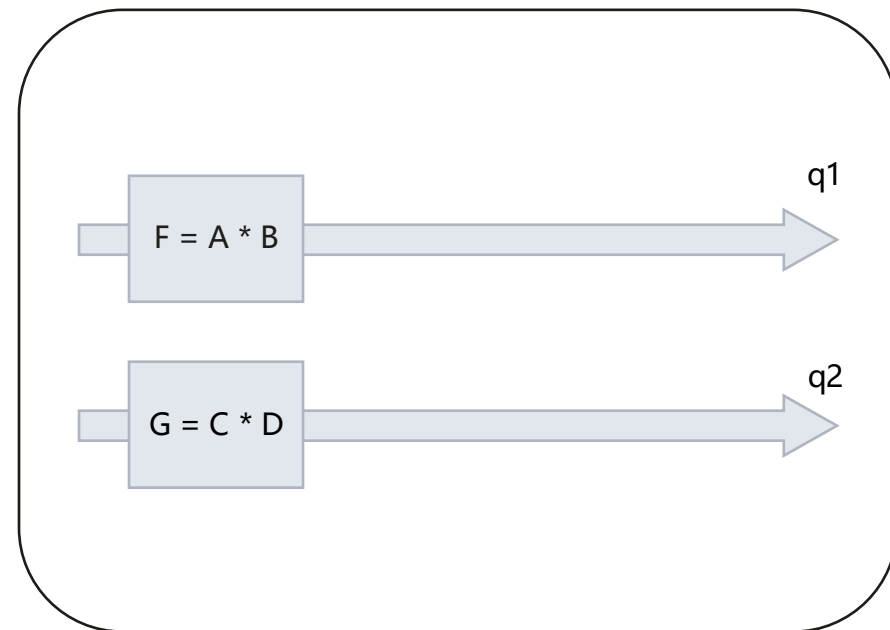


二：众核并行---多队列编程

● 计算表达式 $E = A * B + C * D$ 编程示例

```
auto q1 = kupl_queue_create();

// Step1: 往队列 q1, q2 上添加计算任务
typedef struct data_axb {
    int *F_data; int *A_data; int *B_data;
} data_axb_t;
static void fun_axb(void *args) {
    data_axb_t *data = (data_axb_t *)args;           // 对入参进行解包操作
    int *F = data->F_data, *A = data->A_data, *B = data->B_data;
    (*F) = (*A) * (*B);
}
data_axb_t data1 = {
    .F_data = &F, .A_data = &A, .B_data = &B
};
kupl_queue_item_desc_t desc1 = {
    .func = fun_axb,           // 函数指针赋值
    .args = &data1,           // 对入参进行打包操作
    .name = "axb",
};
kupl_queue_submit(q1, &desc1);
... // 同上往队列 q2 上添加计算任务
kupl_queue_submit(q2, &desc2);
```

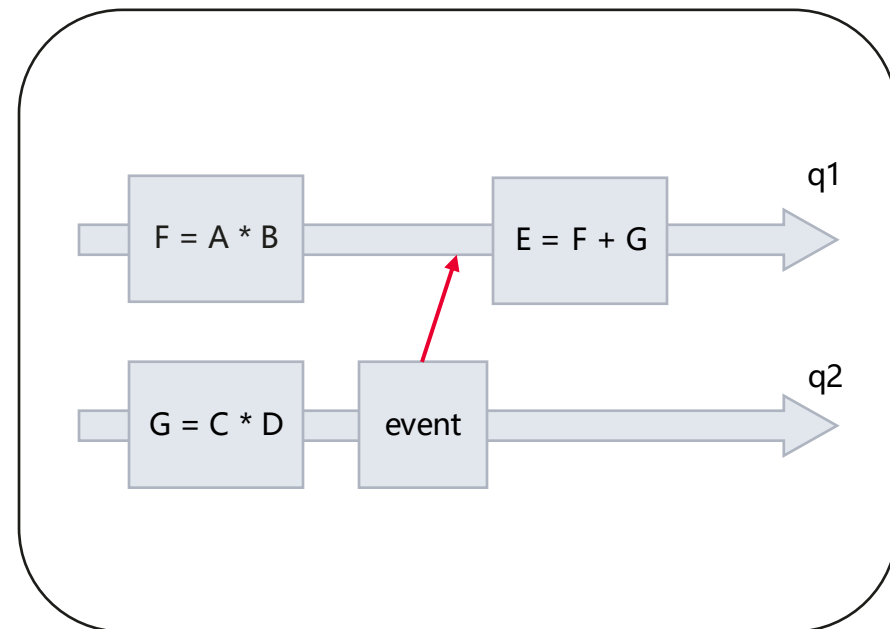


通过kupl_queue_submit接口分别向两个队列各自提交一个任务

二：众核并行---多队列编程

● 计算表达式 $E = A * B + C * D$ 编程示例

```
auto event = kupl_event_create();  
  
// Step2: 往队列 q2 上记录事件  
kupl_event_record(event, q2);  
  
// Step3: 队列 q1, q2 同步  
kupl_queue_wait_event(q1, event);  
  
// Step4: 往队列 q1 上添加计算任务  $E = F + G$   
...  
kupl_queue_submit(q1, &desc3);  
  
// Step5: 等待 q1 上任务执行结束, 打印结果  
kupl_queue_wait(q1);  
printf("E = A * B + C * D, E = %d\n", E);  
  
kupl_event_destroy(event);  
kupl_queue_destroy(q2);  
kupl_queue_destroy(q1);
```



通过event的定义与使用实现q1, q2间的依赖定义

二：众核并行---多队列编程

● 多队列多流相关函数接口

描述	接口名称
创建/销毁队列	kupl_queue_create/kupl_queue_destroy
等待队列执行完毕	kupl_queue_wait
向静态图中添加任务依赖	kupl_queue_wait_event
向队列提交需要执行的 item	kupl_queue_submit
创建/销毁事件	kupl_event_create/kupl_event_destroy
将事件记录在队列上	kupl_event_record
等待事件出于完毕状态	kupl_event_wait
查询事件状态	kupl_event_query

计算图相关函数接口鲲鹏社区链接：

https://www.hikunpeng.com/document/detail/zh/kunpenghpcs/hpckit/devg/kupl_004.html

计算图代码Sample链接：

<https://gitee.com/openeuler/kupl-sample>

其他加速库

BoostKit加速库：支持7类加速库，覆盖主流应用场景需求



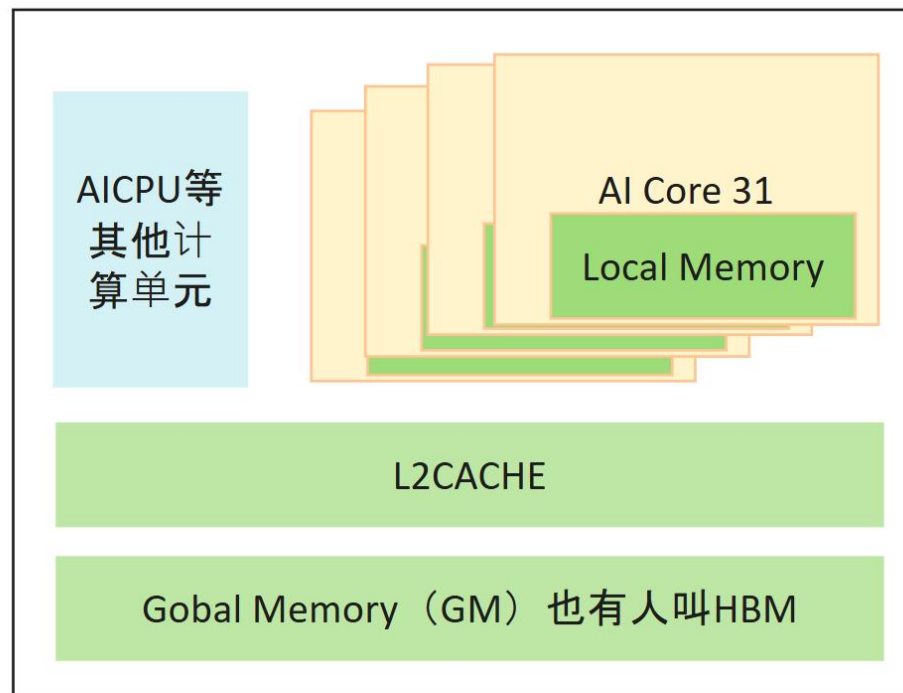
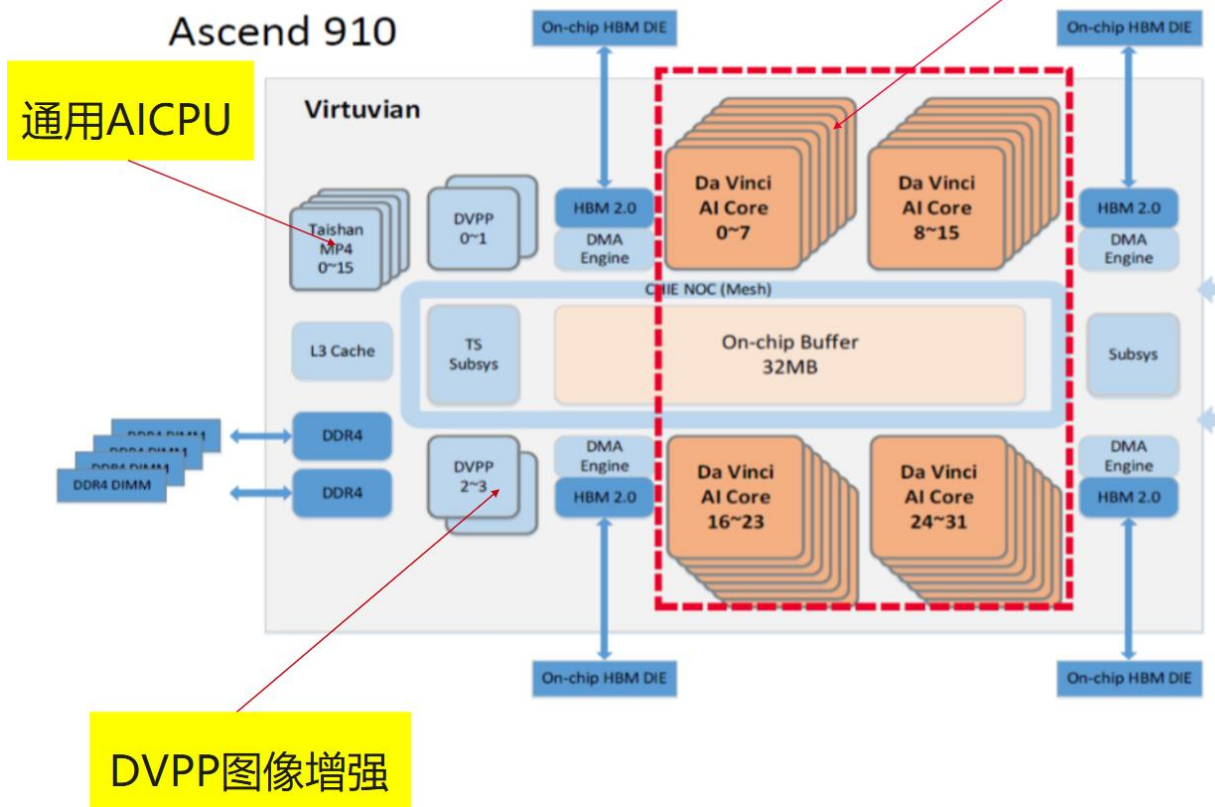
■ 基于ARM指令的软加速库 ■ 基于KAE的硬加速库：包含ZIP(硬加速-压缩)，HPRE（硬加速-非对称加解密），SEC(硬加速-对称加解密)

备注：KAE（Kunpeng Accelerator Engine）鲲鹏加速引擎，HPRE（High Performance RSA Engine）高性能RSA加速引擎，SEC（Security Engine）硬件安全加速引擎

昇腾 - Ascend C编程

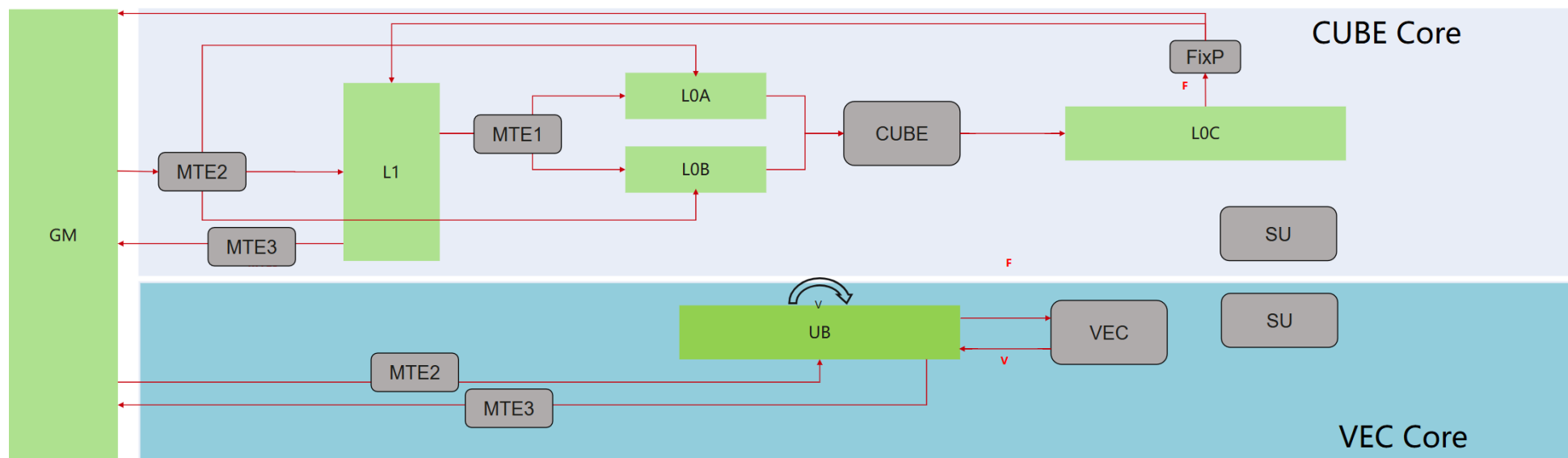
Ascend芯片内部结构

AI CORE, 算力的最核心支撑部件, 有多个核心



内存层次

架构模型



1. CUBE核架构:

1. 5个并行执行单元: MTE1、MTE2、MTE3、CUB、SU
2. 多个内存单元: GM、L2 (透明, 192MB)、L1 (512KB)、L0A/B (64KB)、LOC (128KB)

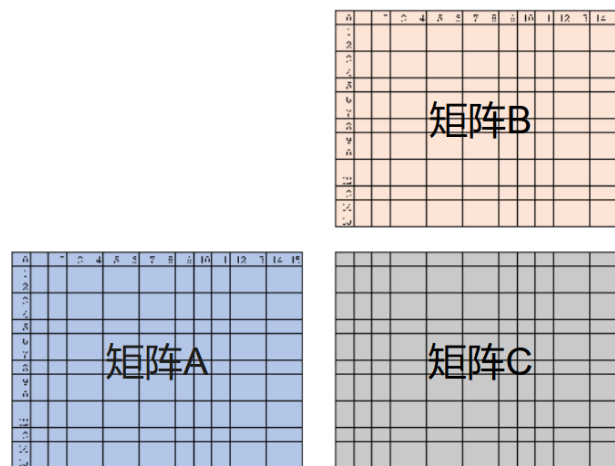
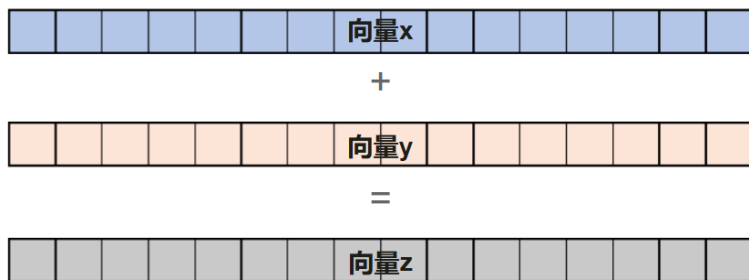
2. VEC核架构

1. 4个并行执行单元: MTE2、MTE3、VEC、SU
2. 2个内存单元: GM、UB

3. 典型计算数据流

1. CUBE: GM -> L1 -> L0A/B -> CUBE -> LOC -> FIXPIPE -> GM
2. VEC: GM -> UB -> VEC -> UB -> GM

AICORE内部支持核心计算：标量、向量、矩阵



标量计算

```
int x = 0, y = 0;
```

```
int z = x + y
```

向量计算，NPU 1个核可以在1个时钟周期

处理128个FP16加法

```
int x[1024], y[1024], z[1024];
```

...

```
Add(z, x, y, 1024)
```

矩阵计算 $C = A * B$

NPU 1个核可以在1个时钟周期处理

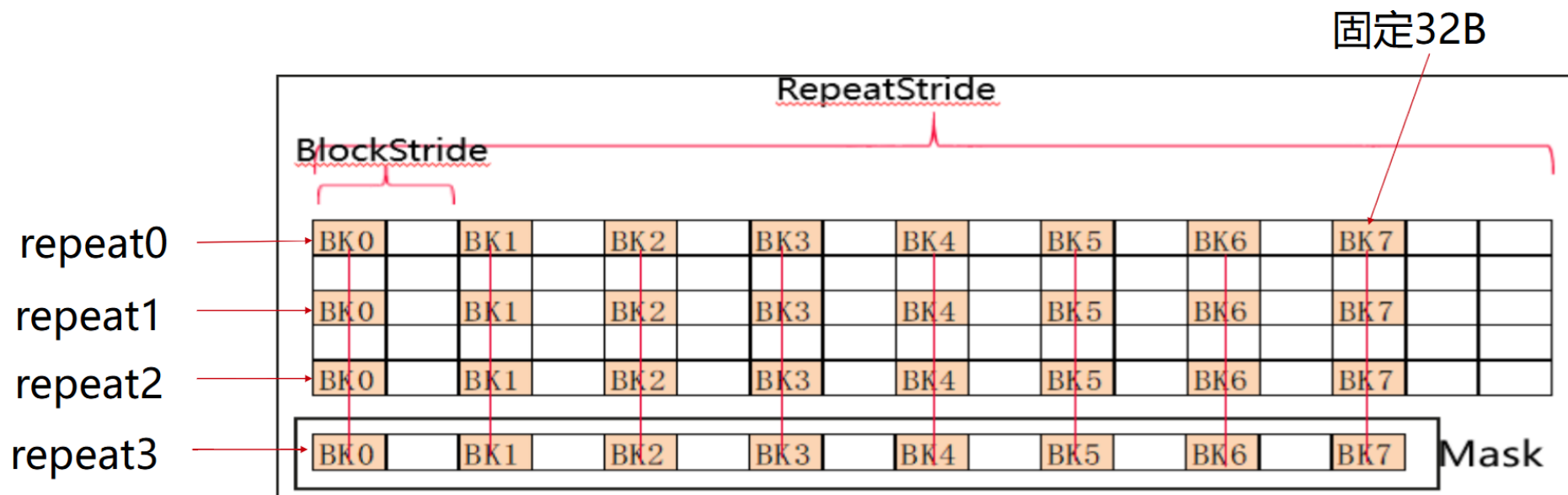
16*16*16的FP16矩阵乘法

```
Mmad(C, A, B, M, K, N)
```

SIMD指令基础知识 – 0级API指令介绍

abs (src, dst, repeatnum, repeatstride, mask, srcblkstr, dstblkstr)

- 内部SIMD计算是按照固定32B * 8 = 256B的宽度进行1次repeat的计算（1个cycle）
- 需要指定src和dst的block stride
- 指定repeat stride（32B为单位）和repeate num
- 指定1个128bits的mask，去控制实际要计算的位置



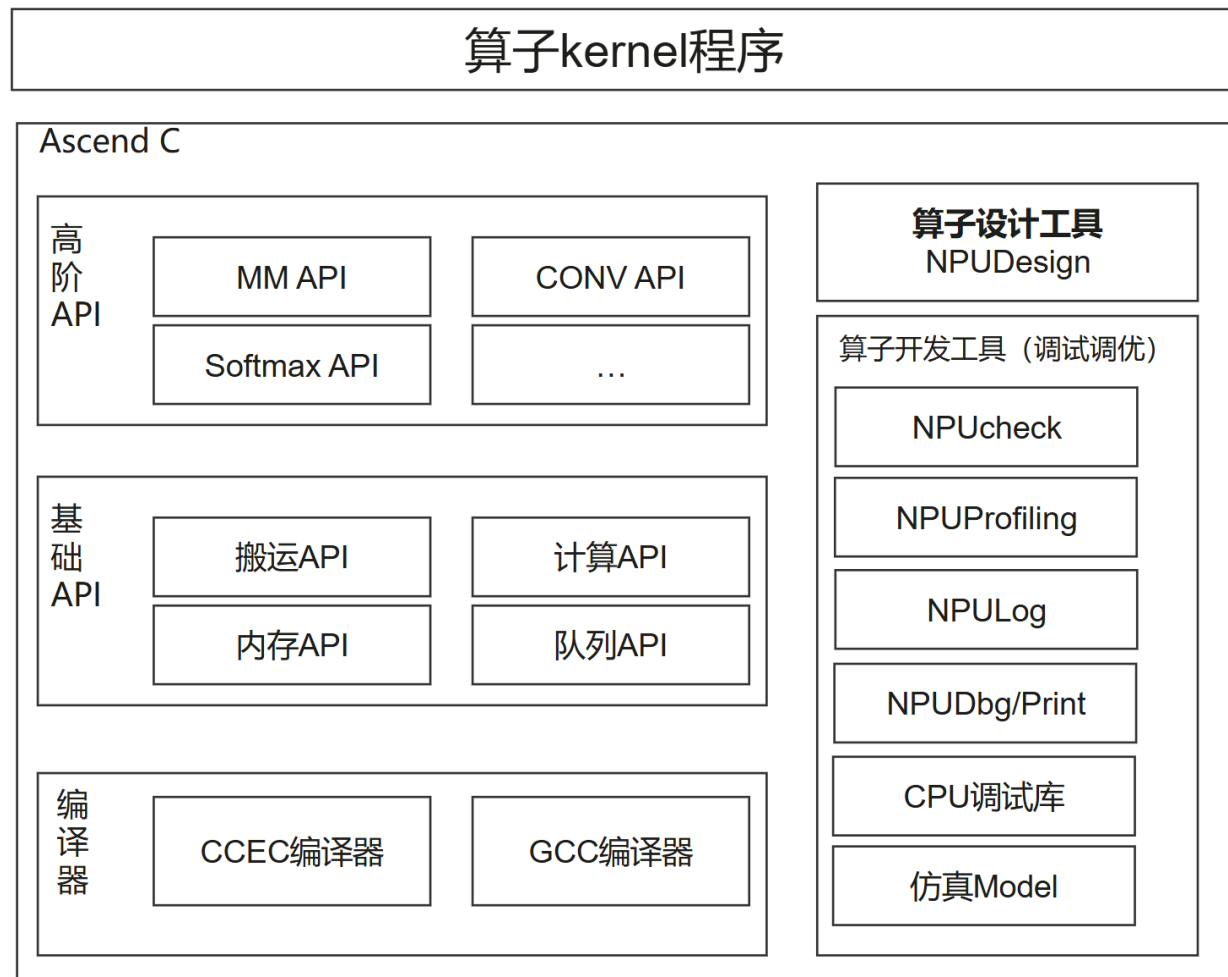
Ascend C 架构组成

Ascend C类库基本API

- 计算：向量计算API、矩阵计算API
- 搬运API：DataCopy
- 同步API：Que API、Buffer API

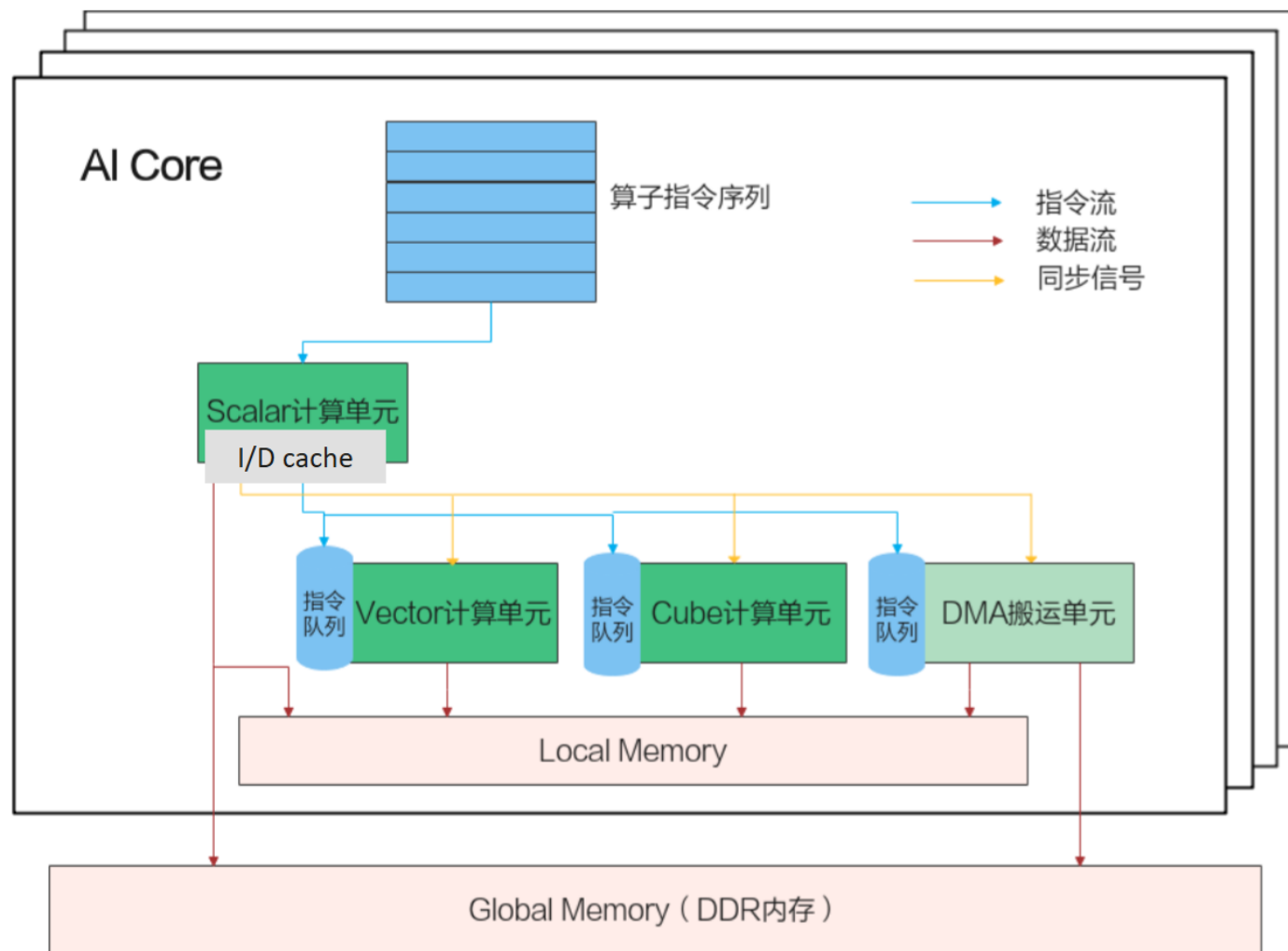
Ascend C类库高阶API

- Matmul
- Convolution
-



AI Core内部并行计算架构抽象

以Scalar计算单元为起点，可以同时向Vector、Cube、DMA单元发射，且Vector、Cube、DMA可并行执行



VEC编程范式：算子框架类实现的一个 $z = x + y$ 算子样例

```
4: #include "kernel_operator.h"
5: using namespace tik2;
6: constexpr int32_t totallength = 8 * 2048; // 总数据长度
7: constexpr int32_t useCoreNum = 8; // 计算中使用的Core数量
8: constexpr int32_t blocklength = totallength / useCoreNum; // 单个Core处理的数据量
9: constexpr int32_t tileNum = 8; // 每个Core上的切分数量
10: constexpr int32_t tilelength = blocklength / tileNum; // 单个Tile块的长度
11: constexpr int32_t bufferNum = 1; // Queue上的Tensor数量

12: class KernelAdd {
13: public:
14:     __aicore__ void Init(__gm__ uint8_t* x, __gm__ uint8_t* y, __gm__ uint8_t* z)
15:     {
16:         xGm.SetGlobalBuffer((__gm__ half*)x + block_idx * blocklength); // 获得本block要计算的数据位置
17:         yGm.SetGlobalBuffer((__gm__ half*)y + block_idx * blocklength);
18:         zGm.SetGlobalBuffer((__gm__ half*)z + block_idx * blocklength);
19:         pipe.InitBuffer(inQueueX, bufferNum, tilelength * sizeof(half)); // 初始化队列的内存资源
20:         pipe.InitBuffer(inQueueY, bufferNum, tilelength * sizeof(half));
21:         pipe.InitBuffer(outQueueZ, bufferNum, tilelength * sizeof(half));
22:     }
23:     __aicore__ void Process()
24:     {
25:         constexpr int32_t loopCount = tileNum;
26:         for (int32_t i = 0; i < loopCount; i++) {
27:             CopyIn(i);
28:             Compute(i);
29:             CopyOut(i);
30:         }
31:     private:
32:     __aicore__ void CopyIn(int32_t progress)
33:     {
34:         LocalTensor<half> xLocal = inQueueX.AllocTensor<half>(); // 获得tensor所需内存资源
35:         LocalTensor<half> yLocal = inQueueY.AllocTensor<half>();
36:         DataCopy(xLocal, xGm[progress * tilelength], tilelength); // 复制当前第progress分片到local
37:         DataCopy(yLocal, yGm[progress * tilelength], tilelength);
38:         inQueueX.Enqueue(xLocal);
39:         inQueueY.Enqueue(yLocal);
40:     }
41:     __aicore__ void Compute(int32_t progress)
42:     {
43:         LocalTensor<half> xLocal = inQueueX.DeQueue<half>();
44:         LocalTensor<half> yLocal = inQueueY.DeQueue<half>();
45:         LocalTensor<half> zLocal = outQueueZ.AllocTensor<half>();
46:         Add(zLocal, xLocal, yLocal, tilelength);
47:         outQueueZ.Enqueue<half>(zLocal);
48:         inQueueX.FreeTensor(xLocal); // 释放tensor
49:         inQueueY.FreeTensor(yLocal);
50:     }
51:     __aicore__ void CopyOut(int32_t progress)
52:     {
53:         LocalTensor<half> zLocal = outQueueZ.DeQueue<half>();
54:         DataCopy(zGm[progress * tilelength], zLocal, tilelength);
55:         outQueueZ.FreeTensor(zLocal);
56:     }
57:     private:
58:     TPipe pipe;
59:     TQue<QuePosition::VECIN, bufferNum> inQueueX, inQueueY; // 创建输入队列资源
60:     TQue<QuePosition::VECOUT, bufferNum> outQueueZ; // 创建输出队列资源
61:     GlobalTensor<half> xGm, yGm, zGm;
62: } // end KernelAdd
63: extern "C" __global__ __aicore__ void add_kernel(__gm__ uint8_t* x, __gm__ uint8_t* y, __gm__ uint8_t* z)
64: {
65:     KernelAdd op;
66:     op.Init(x, y, z);
67:     op.Process();
68: }
```

1, 分配队列的内存资源

2, 完成copyin Stage的处理

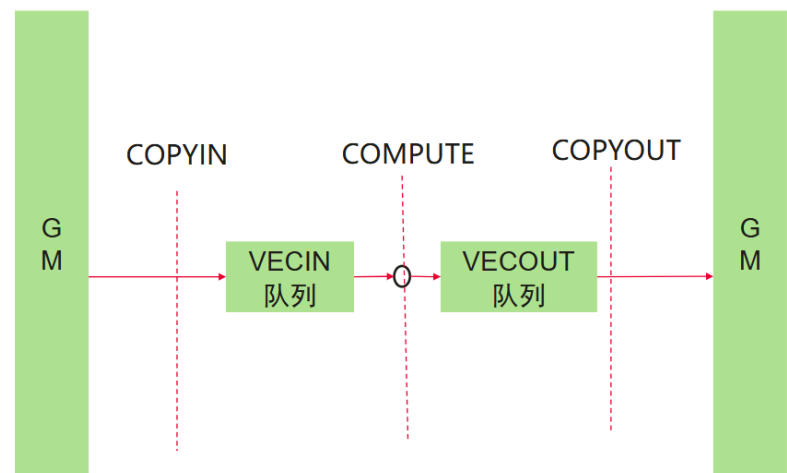
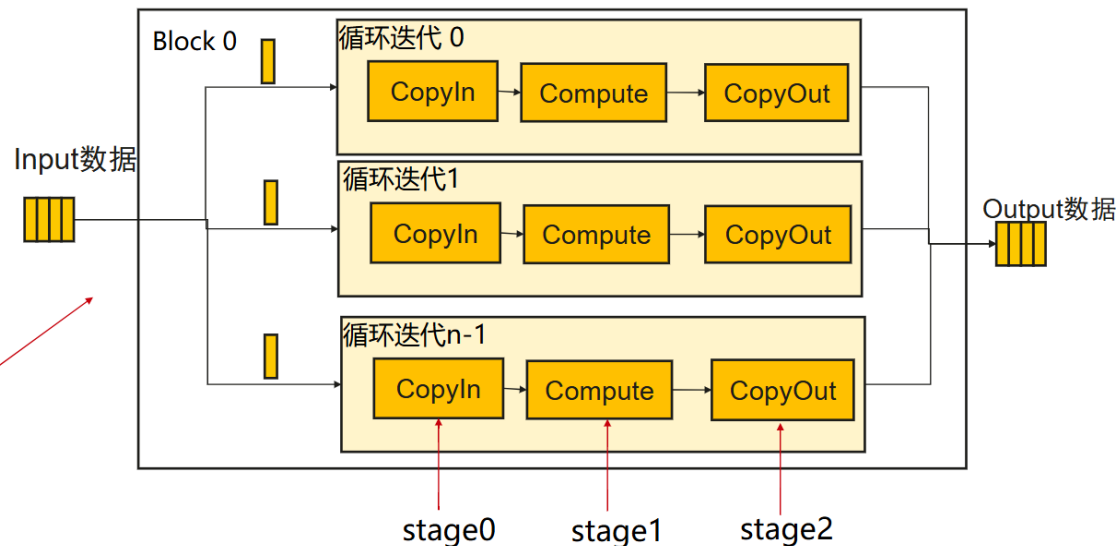
3, 完成compute Stage的处理

4, 完成copyout Stage的处理

5, 算子调度程序

0, 定义输入输出队列

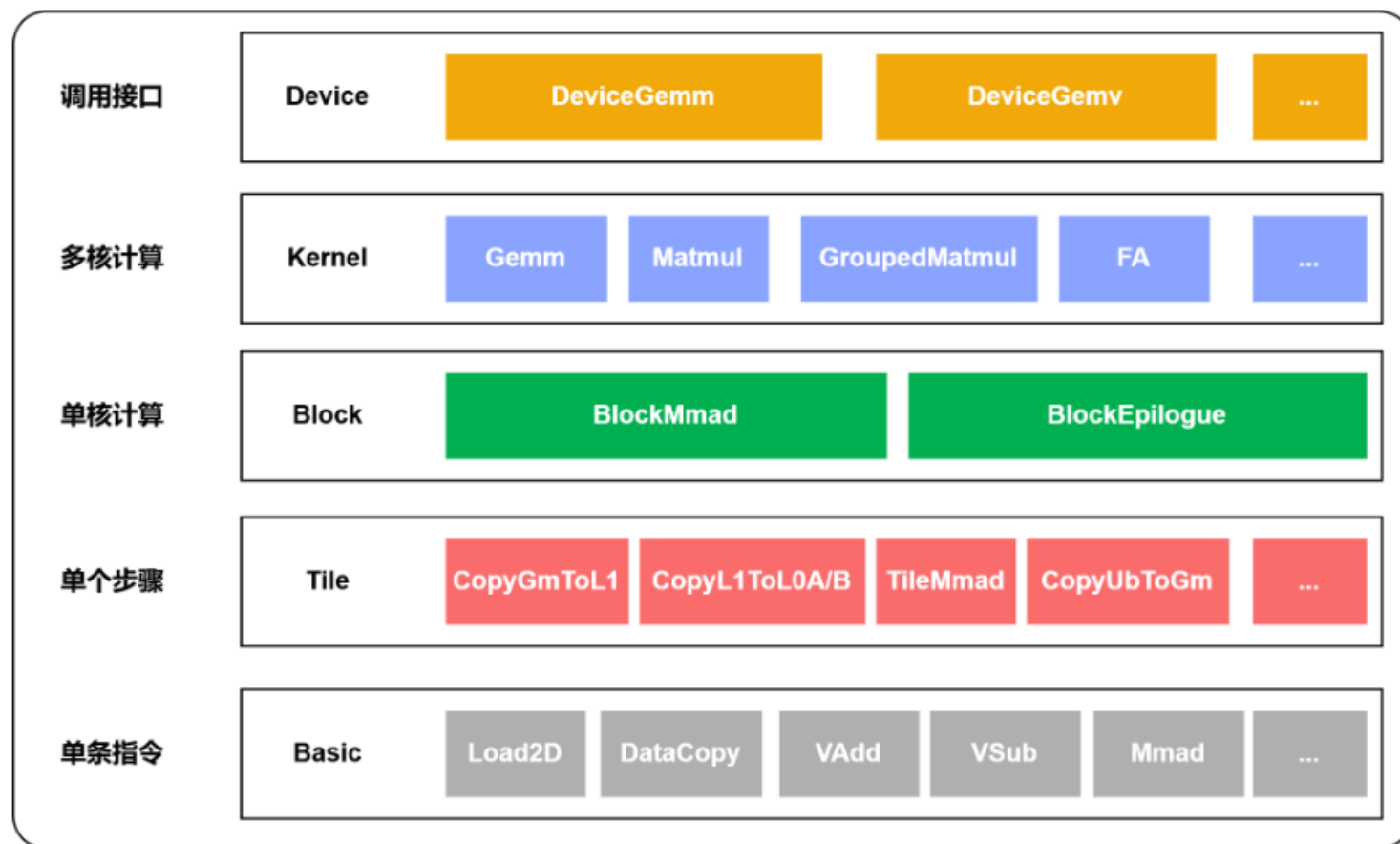
算子的主程序



矩阵乘法模板库

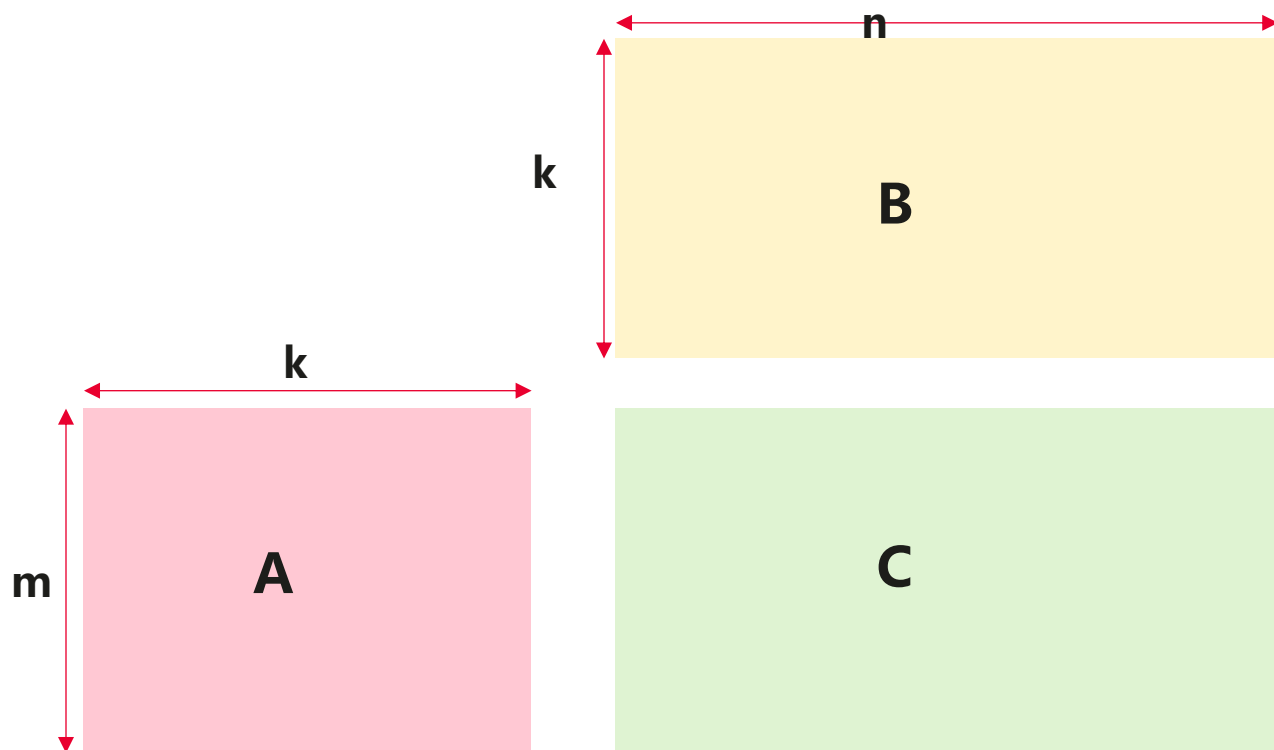
<https://gitee.com/ascend/ascendc-templates>

对标nvidia cutlass, 矩阵乘法模板库,
降低高性能矩阵乘法类算子开发门槛



优化样例 – 矩阵乘法

样例：单精度矩阵乘法



1. 数据布局:

- A: Column Major
- B: Row Major
- C: Row Major

2. 数据规模

- $m=n=k=2048$
- $m=n=k=4096$
- $m=n=k=8192$

3. 硬件:

- 鲲鹏920 单核: 41.6 GFlops
- 鲲鹏920 单numa: 1331 GFlops

4. 编译器:

- 毕昇C++

优化1~3：调整循环顺序，优化访存的连续性

```
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        float sum = 0;
        for (int kk = 0; kk < k; kk++) {
            sum += a[kk*lda+i] * b[kk*ldb+j];
        }
        c[i*ldc+j] = beta * c[i*ldc+j] + alpha * sum;
    }
}
```

m - n - k

- 劣势：a和b访存不连续
- 优势：c访存连续、c 1读1写

```
float *buf_ = AlignMalloc<float>(n * tnum);
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) buf[j] = 0;
    for (int kk = 0; kk < k; kk++) {
        for (int j = 0; j < n; j++) {
            buf[j] += a[kk*lda+i] * b[kk*ldb+j];
        }
    }
    for (int j = 0; j < n; j++) {
        c[i*ldc+j] = beta * c[i*ldc+j] + alpha * buf[j];
    }
}
free(buf);
```

m - k - n

- 劣势：需要大小为n的额外空间；a的访存不连续
- 优势：b和c的访存连续；c 1读1写

```
float *buf = AlignMalloc<float>(m * n);
for (int i = 0; i < m * n; i++) buf[i] = 0;
for (int kk = 0; kk < k; kk++) {
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            buf[i*n+j] += a[kk*lda+i] * b[kk*ldb+j];
        }
    }
}
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        c[i*ldc+j] = beta * c[i*ldc+j] + alpha * buf[i*n+j];
    }
}
free(buf);
```

k - m - n

- 劣势：需要大小为m*n的额外空间；c（即buf）需要k读k写
- 优势：a、b、c的访存连续

编号	优化点	2048 serial(GFlops)	4096 serial(GFlops)	8192 serial(GFlops)	2048 parallel(GFlops)	4096 parallel(GFlops)	8192 parallel(GFlops)
1	for循环顺序：m-n-k	0.225	0.225	0.225	1.027	0.874	0.986
2	for循环顺序：m-k-n	1.880	1.612	1.392	56.219	50.295	46.974
3	for循环顺序：k-m-n	6.526	4.531	4.495	268.094	19.056	17.736

优化4：寄存器分块

目的：提高计算访存比 – 更多的FMLA & 更少的LOAD

已知：

- 1个向量寄存器可以存4个float
- 有32个向量寄存器
- 鲲鹏920每个cycle可以吞吐2条FMLA
- FMLA的指令延迟是5

限制：

- 分块后使用的寄存器数量小于等于32
- 尽可能用大于等于5*2个寄存器保存c，以排布FMLA
- 尽可能提高#FMLA/#LD

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

0
0
0
0
1
1
1
1

0	0	0	0	1	1	1	1
2	2	2	2	3	3	3	3
4	4	4	4	5	5	5	5
6	6	6	6	7	7	7	7
8	8	8	8	9	9	9	9
10	10	10	10	11	11	11	11
12	12	12	12	13	13	13	13
14	14	14	14	15	15	15	15

结论：

- 分块大小 $4 * 16$
- C需要16个寄存器、A和B分别需要2个寄存器
- 共需要20个寄存器（考虑双buffer需要24个）
- 计算访存比 $16 / (2 + 2) = 4$

优化4：寄存器分块 – 续

```
float32x4_t cc[BM][BN/4];
for (int i = 0; i < BM; i++)
    for (int j = 0; j < BN/4; j++)
        cc[i][j] = vdupq_n_f32(0);
float32x4_t aa[BM/4];
float32x4_t bb[BN/4];
for (int s = 0; s < length; s++) {
    for (int i = 0; i < BM/4; i++) aa[i] = vld1q_f32(a + i*4);
    a += lda;
    for (int j = 0; j < BN/4; j++) bb[j] = vld1q_f32(b + j*4);
    b += ldb;
    for (int j = 0; j < BN/4; j++) {
        for (int i = 0; i < BM/4; i++) {
            cc[i*4+0][j] = vfmaq_laneq_f32(cc[i*4+0][j], bb[j], aa[i], 0);
            cc[i*4+1][j] = vfmaq_laneq_f32(cc[i*4+1][j], bb[j], aa[i], 1);
            cc[i*4+2][j] = vfmaq_laneq_f32(cc[i*4+2][j], bb[j], aa[i], 2);
            cc[i*4+3][j] = vfmaq_laneq_f32(cc[i*4+3][j], bb[j], aa[i], 3);
        }
    }
}
```

```
constexpr int BM = 8;
constexpr int BN = 8;
for (int i = 0; i < m; i += BM) {
    for (int j = 0; j < n; j += BN) {
        Kernel<BM, BN>(
            k,
            alpha,
            a + i, lda,
            b + j, ldb,
            beta,
            c + i*ldc + j, ldc);
    }
}
```

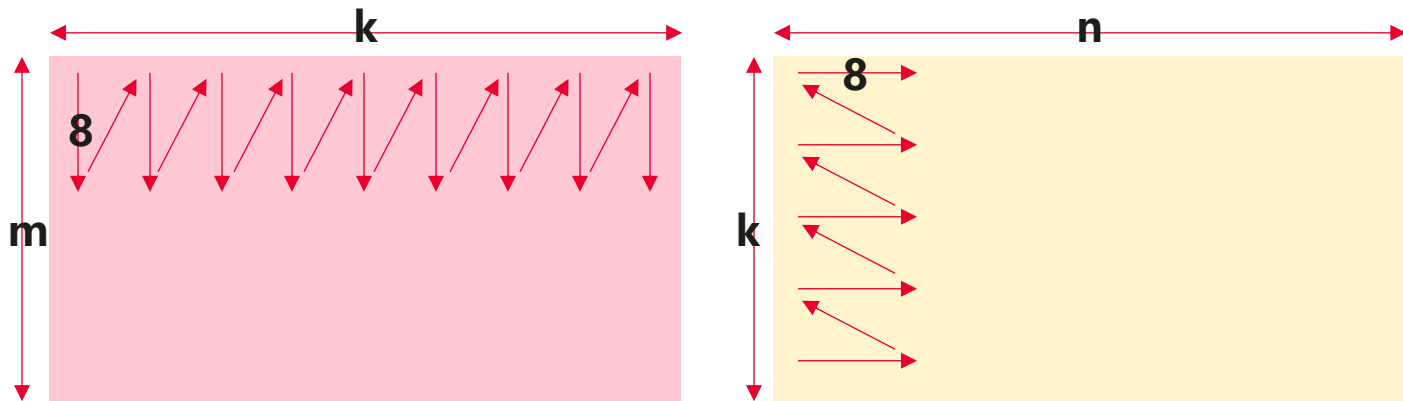
寄存器分块

- 优势：极致提高计算访存比
- 劣势：破坏了A和B的访存连续性（近似于 $m - n - k$ ）

编号	优化点	2048 serial(GFlops)	4096 serial(GFlops)	8192 serial(GFlops)	2048 parallel(GFlops)	4096 parallel(GFlops)	8192 parallel(GFlops)
3	for循环顺序: k-m-n	6.526	4.531	4.495	268.094	19.056	17.736
4	寄存器分块	6.867	3.544	2.527	71.171	45.917	41.888

优化5：重排A和B

目的：优化4又劣化的访存的连续性，通过重排A和B优化



重排A和B

- 优势：A和B的访存连续性
- 劣势：需要额外空间和时间去重排矩阵，适合较大规模矩阵乘法

```
float *bufa = AlignMalloc<float>(m * k);
float *bufb = AlignMalloc<float>(n * k);
constexpr int BM = 8;
constexpr int BN = 8;
Permute<BM>(k, m, a, lda, bufa, tnum);
Permute<BN>(k, n, b, ldb, bufb, tnum);
for (int i = 0; i < m; i += BM) {
    for (int j = 0; j < n; j += BN) {
        KernelV2<BM, BN>(
            k,
            alpha,
            bufa + i*k,
            bufb + j*k,
            beta,
            c + i*ldc + j, ldc);
    }
}
free(bufa);
free(bufb);
```

编号	优化点	2048 serial(GFlops)	4096 serial(GFlops)	8192 serial(GFlops)	2048 parallel(GFlops)	4096 parallel(GFlops)	8192 parallel(GFlops)
4	寄存器分块	6.867	3.544	2.527	71.171	45.917	41.888
5	重排A和B	25.474	17.636	17.515	515.287	523.498	545.221

优化6: Kernel双Buffer

目的: 访存需要时间, 通过双buffer可以预发射访存指令, 通过计算部分掩藏访存

```
Load 0 & 1
for (i = 2; i < length; i += 2) {
    Load 2 & 3
    Use 0 & 1 to Compute
    Load 0 & 1
    Use 2 & 3 to Compute
}
Load 2 & 3
Use 0 & 1 to Compute
Use 2 & 3 to Compute
```

Kernel双Buffer

- 优势: 计算掩藏访存
- 劣势: 需要更多的寄存器

0	2
0	2
0	2
0	2
1	3
1	3
1	3
1	3

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

2	2	2	2	3	3	3	3
---	---	---	---	---	---	---	---

0	0	0	0	1	1	1	1
2	2	2	2	3	3	3	3
4	4	4	4	5	5	5	5
6	6	6	6	7	7	7	7
8	8	8	8	9	9	9	9
10	10	10	10	11	11	11	11
12	12	12	12	13	13	13	13
14	14	14	14	15	15	15	15

编号	优化点	2048 serial(GFlops)	4096 serial(GFlops)	8192 serial(GFlops)	2048 parallel(GFlops)	4096 parallel(GFlops)	8192 parallel(GFlops)
5	重排A和B	25.474	17.636	17.515	515.287	523.498	545.221
6	Kernel双buffer	28.371	20.089	19.355	517.234	517.455	542.816

优化7：汇编优化

长度减2，并更新条件标志位
FMLA指令和LD指令排布在一起

目的：通过手写汇编严格执行双
buffer+prefetch+指令排布

加入prefetch指令

Kernel双Buffer

- 优势：精准极致地控制计算流程和寄存器
- 劣势：难以维护

```
"subs    %[len], %[len], #2 \n"
"fmla    %[cc00].4s, v24.4s, v28.s[0] \n"
"fmla    %[cc10].4s, v24.4s, v28.s[1] \n"
"ldl     {v26.4s}, [%[ap]], #16 \n"
"fmla    %[cc20].4s, v24.4s, v28.s[2] \n"
"fmla    %[cc30].4s, v24.4s, v28.s[3] \n"
"ldl     {v30.4s}, [%[bp]], #16 \n"
"fmla    %[cc01].4s, v25.4s, v28.s[0] \n"
"fmla    %[cc11].4s, v25.4s, v28.s[1] \n"
"ldl     {v27.4s}, [%[ap]], #16 \n"
"fmla    %[cc21].4s, v25.4s, v28.s[2] \n"
"fmla    %[cc31].4s, v25.4s, v28.s[3] \n"
"ldl     {v31.4s}, [%[bp]], #16 \n"
"fmla    %[cc40].4s, v24.4s, v29.s[0] \n"
"fmla    %[cc50].4s, v24.4s, v29.s[1] \n"
"prfm    pldl1strm, [%[ap]], #256] \n"
"fmla    %[cc60].4s, v24.4s, v29.s[2] \n"
"fmla    %[cc70].4s, v24.4s, v29.s[3] \n"
"prfm    pldl1keep, [%[bp]], #256] \n"
"fmla    %[cc41].4s, v25.4s, v29.s[0] \n"
"fmla    %[cc51].4s, v25.4s, v29.s[1] \n"
"fmla    %[cc61].4s, v25.4s, v29.s[2] \n"
"fmla    %[cc71].4s, v25.4s, v29.s[3] \n"
"ldl     {v24.4s}, [%[ap]], #16 \n"
"fmla    %[cc00].4s, v26.4s, v30.s[0] \n"
"fmla    %[cc10].4s, v26.4s, v30.s[1] \n"
"ldl     {v28.4s}, [%[bp]], #16 \n"
"fmla    %[cc20].4s, v26.4s, v30.s[2] \n"
"fmla    %[cc30].4s, v26.4s, v30.s[3] \n"
"ldl     {v25.4s}, [%[ap]], #16 \n"
"fmla    %[cc01].4s, v27.4s, v30.s[0] \n"
"fmla    %[cc11].4s, v27.4s, v30.s[1] \n"
"ldl     {v29.4s}, [%[bp]], #16 \n"
"fmla    %[cc21].4s, v27.4s, v30.s[2] \n"
"fmla    %[cc31].4s, v27.4s, v30.s[3] \n"
"fmla    %[cc40].4s, v26.4s, v31.s[0] \n"
"fmla    %[cc50].4s, v26.4s, v31.s[1] \n"
"fmla    %[cc60].4s, v26.4s, v31.s[2] \n"
"fmla    %[cc70].4s, v26.4s, v31.s[3] \n"
"fmla    %[cc41].4s, v27.4s, v31.s[0] \n"
"fmla    %[cc51].4s, v27.4s, v31.s[1] \n"
"fmla    %[cc61].4s, v27.4s, v31.s[2] \n"
"fmla    %[cc71].4s, v27.4s, v31.s[3] \n"
"bne     loop %= \n"
```

如果length > 0则跳到第一行

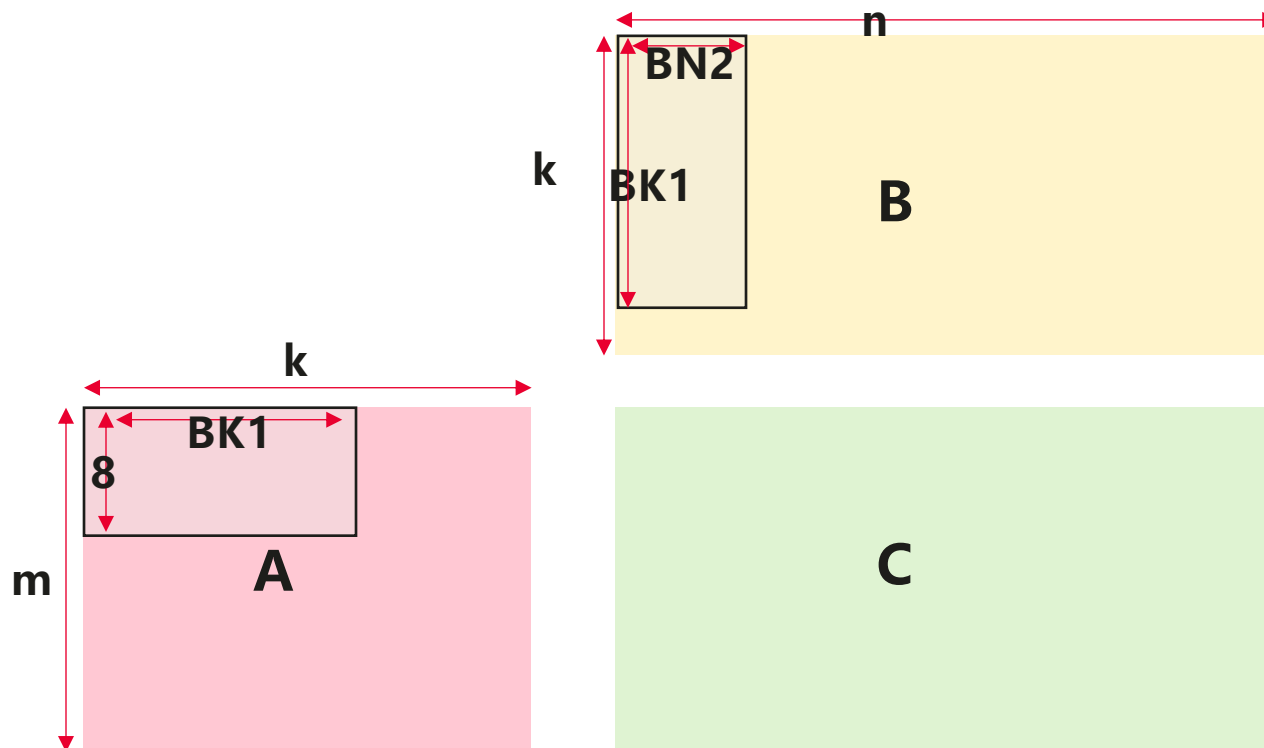
编号	优化点	2048 serial(GFlops)	4096 serial(GFlops)	8192 serial(GFlops)	2048 parallel(GFlops)	4096 parallel(GFlops)	8192 parallel(GFlops)
6	Kernel双buffer	28.371	20.089	19.355	517.234	517.455	542.816
7	Kernel汇编	29.985	23.320	23.244	530.479	567.659	576.148

优化8~9: Cache Blocking

目的：对计算流程进行分块，使得访存的数据尽可能在Cache中，以节省内存带宽

优化思路：

- $\text{sizeof(double)} * (8+8) * \text{BK1} \leq \text{L1 Cache}$
- $\text{sizeof(double)} * \text{BK1} * \text{BK2} \leq \text{L2 Cache}$



优化8~9: Cache Blocking – 续

目的：对计算流程进行分块，使得访存的数据尽可能在Cache中，以节省内存带宽

Cache Blocking:

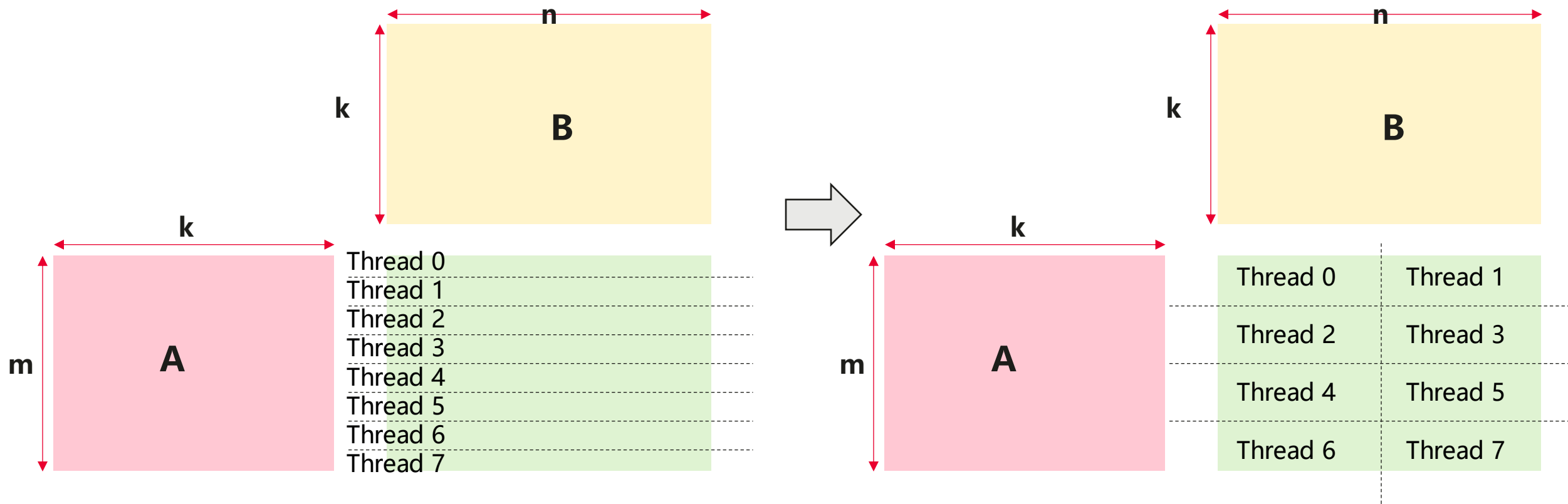
- 选择BK1=512, BN2=128

```
int BK1 = 512;
float *bufa = AlignMalloc<float>(m * BK1);
float *bufb = AlignMalloc<float>(n * BK1);
constexpr int BM = 8;
constexpr int BN = 8;
int BN2 = 128;
assert(BN2%BN == 0);
float beta_ = beta;
for (int kk = 0; kk < k; kk += BK1) {
    int klength = std::min(BK1, k - kk);
    Permute<BM>(klength, m, a + kk*lda, lda, bufa, tnum);
    Permute<BN>(klength, n, b + kk*ldb, ldb, bufb, tnum);
    for (int jj = 0; jj < n; jj += BN2) {
        int nlength = std::min(BN2, n - jj);
        for (int i = 0; i < m; i += BM) {
            for (int j = 0; j < nlength; j += BN) {
                KernelV4<BM, BN>(
                    klength,
                    alpha,
                    bufa + i*klength,
                    bufb + (jj+j)*klength,
                    beta_,
                    c + i*ldc + (jj+j), ldc);
            }
        }
    }
    beta_ = 1.0f;
}
free(bufa);
free(bufb);
```

编号	优化点	2048 serial(GFlops)	4096 serial(GFlops)	8192 serial(GFlops)	2048 parallel(GFlops)	4096 parallel(GFlops)	8192 parallel(GFlops)
7	Kernel汇编	29.985	23.320	23.244	530.479	567.659	576.148
8	L2 Cache B	29.157	28.894	25.677	540.584	551.786	507.202
9	L1 Cache A	37.919	38.007	38.126	945.230	1059.298	1121.839

优化10：二维分块

目的：通过二维排布线程，提升数据复用



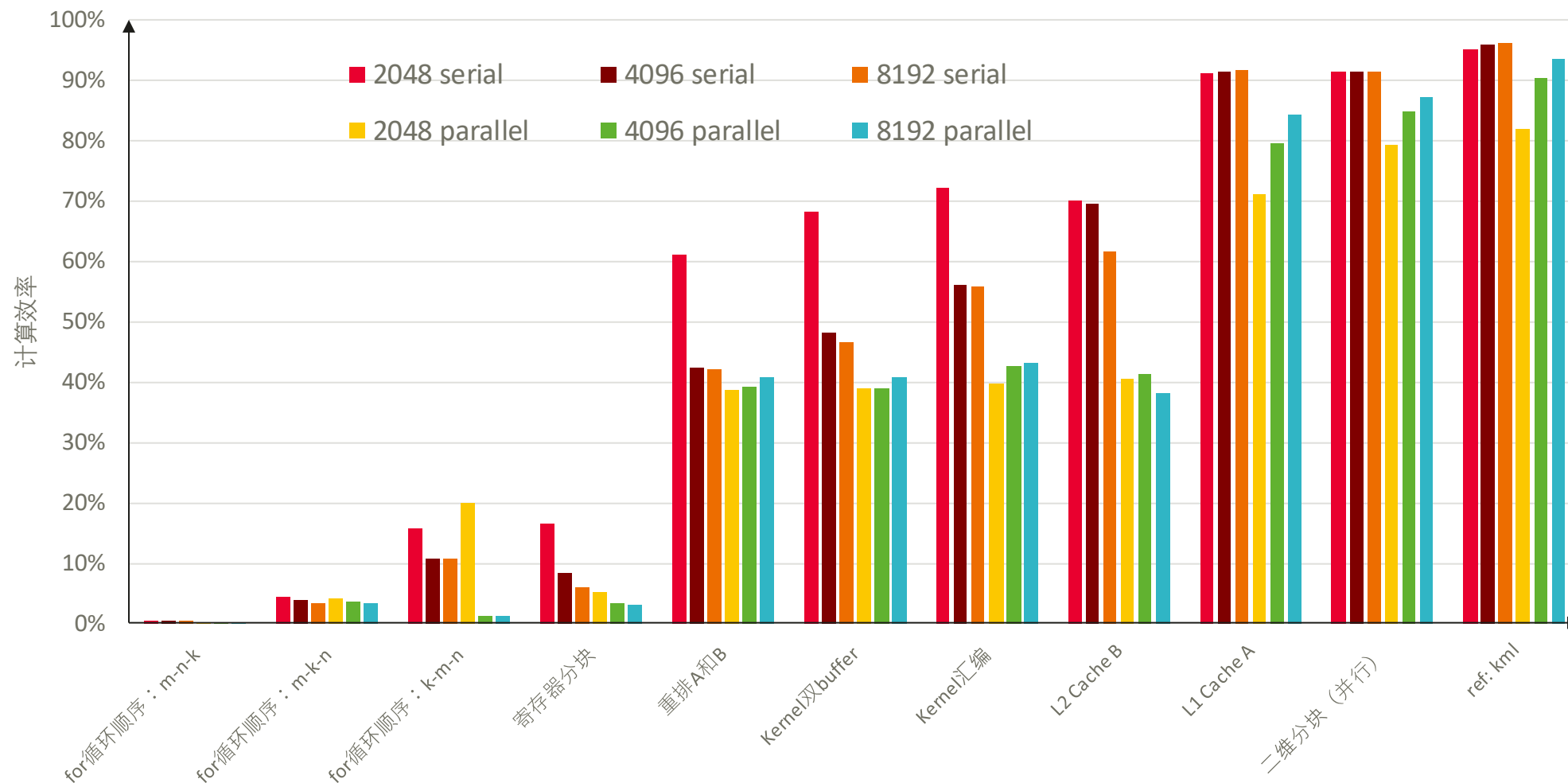
编号	优化点	2048 serial(GFlops)	4096 serial(GFlops)	8192 serial(GFlops)	2048 parallel(GFlops)	4096 parallel(GFlops)	8192 parallel(GFlops)
9	L1 Cache A	37.919	38.007	38.126	945.230	1059.298	1121.839
10	二维分块 (并行)	38.080	38.042	38.050	1055.784	1129.304	1162.043

性能汇总

单位: GFlops

编号	优化点	2048 serial(GFlops)	4096 serial(GFlops)	8192 serial(GFlops)	2048 parallel(GFlops)	4096 parallel(GFlops)	8192 parallel(GFlops)
0	ref: openblas	28.105	28.273	28.415	829.443	865.497	385.307
1	for循环顺序: m-n-k	0.225	0.225	0.225	1.027	0.874	0.986
2	for循环顺序: m-k-n	1.880	1.612	1.392	56.219	50.295	46.974
3	for循环顺序: k-m-n	6.526	4.531	4.495	268.094	19.056	17.736
4	寄存器分块	6.867	3.544	2.527	71.171	45.917	41.888
5	重排A和B	25.474	17.636	17.515	515.287	523.498	545.221
6	Kernel双buffer	28.371	20.089	19.355	517.234	517.455	542.816
7	Kernel汇编	29.985	23.320	23.244	530.479	567.659	576.148
8	L2 Cache B	29.157	28.894	25.677	540.584	551.786	507.202
9	L1 Cache A	37.919	38.007	38.126	945.230	1059.298	1121.839
10	二维分块 (并行)	38.080	38.042	38.050 (91.5%)	1055.784	1129.304	1162.043(87.3%)
11	ref: kml	39.597	39.852	39.951(96.0%)	1090.963	1201.823	1244.166(93.5%)

性能汇总：续



经过优化，单核性能提高**177**倍，多核性能提高**1260**倍

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home and
organization for a fully connected,
intelligent world.

**Copyright©2018 Huawei Technologies Co., Ltd.
All Rights Reserved.**

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

